End-to-End Autonomous Navigation using Homography-Based Overhead Localization

John Christian Niño T. Abuel
Rey Iann Tigley
Kervin Lemuel Paalisbo

Supervised by
Asst Prof. Renato V. Crisostomo, MSCS

Department of Computer Science
College of Computer Studies
MSU - Iligan Institute of Technology

Jan, 2026

A undergraduate thesis submitted in partial fulfilment of the requirements for the degree of B.S. in Computer Science.

www.msuiit.edu.ph

Abstract

Traditional autonomous navigation systems usually depend on expensive multi-sensor fusion or computationally intensive deep learning architectures that process raw, ego-centric video feeds. These high resource requirements create constraints to deploying scalable, low-cost robotics platforms in educational or industrial settings. To address these computational and hardware limitations, a lightweight end-to-end navigation framework is proposed that effectively bridges geometric computer vision with imitation learning. The architecture utilizes a single RGB camera and ground-plane fiducial markers to dynamically generate a homography matrix, transforming perspective-distorted frames into rectified, top-down Birds-Eye-View (BEV) images. These geometrically simplified inputs are subsequently fed into a streamlined Convolutional Neural Network (CNN) trained via behavioral cloning to map path features directly to continuous steering angles. The system is rigorously evaluated across four track topologies of increasing geometric difficulty—45-degree Bézier curves, a 90-degree square path, sustained 180-degree U-turns, and a held-out track unseen during training—to assess closed-loop stability and generalization without explicit PID feedback. Performance is quantified using Root Mean Square Error (RMSE) for cross-track accuracy, heading error for orientation, and per-scenario success rate over ten independent randomized-start trials each. The controller completed all 40 trials without leaving the lane, and tracking error on the unseen topology grew only modestly (about 1.7 times the trained-track RMSE) while reliability was maintained, indicating that the network learned transferable relative path geometry rather than memorized coordinates. The approach demonstrates that integrating classical homography techniques with deep learning significantly reduces computational overhead, offering a feasible alternative to heavy SLAM-based methods for structured indoor environments.

Keywords: Autonomous Navigation, Convolutional Neural Networks, Homography, Behavioral Cloning, Mobile Robots, Computer Vision

Contents

1

Introduction

Rather than relying on manually tuned classical controllers, this study implements an end-to-end control strategy where a single neural network is responsible for the entire navigation pipeline, from perception to actuation. By preprocessing the environment using ArUco-based localization, the 'intelligence' is shifted from complex feature extraction to direct motion planning. This eliminates the latency and complexity associated with tuning PID gains, resulting in a potentially better architecture where the robot learns to interpret geometric spatial relationships and execute smooth motor responses simultaneously.

1.1 | Background of the Study

Autonomous navigation is the ability of robots and autonomous vehicles to generate a path and move along it without human intervention, utilizing data captured from onboard sensors and sometimes remote navigation aids Cebollada et al. (2020). These systems consistently rely on simultaneous localization and mapping (SLAM) for autonomous navigation. Additionally, these systems also utilize LiDAR, depth cameras, or multi-sensor fusion Liu et al. (2024); Wei et al. (2024); Zhu et al. (2023). While generally effective, these robotic systems demand high computational costs, expensive robotics components, and complex software integration requirements, which makes them less accessible for developing low-cost robotics platforms Chea et al. (2023). To address these limitations, previous studies have already explored computer-vision navigation using fiducial markers, which offer a simple, lightweight, and reliable framework for localization, particularly in structured environments Alghamdi et al.

(2025).

According to Kuzmenko (2025), a fiducial marker refers to a specialized code-type marker used in computer vision, which is set in an environment or scene to help imaging systems find reference points. The robustness of fiducial markers such as ArUco and AprilTag in providing low-cost localization has already been explored in a 2025 systematic review of the mapping of mobile robot navigation Alghamdi et al. (2025). The study of Mantha and De Soto (2022) concluded that marker size, spacing, and the placement of a fiducial marker may affect robot navigation, well-designed markers show potential in achieving high success rates with minimal computational overhead.

Several recent studies probed the ability of ArUco markers to provide indoor localization for autonomous robot navigation. Research on planar indoor localization using ArUco Hinderer et al. (2025) and multi-marker setups illustrates that planar marker fields can deliver accurate 2-D pose estimates which are appropriate for autonomous robot navigation Lin (2023), especially when combined with simple filters or odometry. To further maximize localization, multi-camera setups are also explored, as seen in the study of Sevgi and Koçer (2025). Studies on fiducial-marker networks for indoor navigation Braun et al. (2022) explicitly frame markers as an alternative to the expensive SLAM for building service and inspection robots, highlighting low instrumentation and computation as key advantages.

For robot navigation, recent research has shown interest in using end-to-end frameworks that map visual input directly to steering Han et al. (2024) or obstacle avoidance actions, usually with deep reinforcement learning or attention-based architecture Yu et al. (2025); Zhou and Zhang (2025). Although such methods can achieve strong obstacle avoidance performance in simulation or structured test environments, multiple studies emphasize issues with stability, overfitting, and susceptibility to noise, leading to poor generalization outside training conditions.

Current research in End-to-End Visual Navigation (e.g., NVIDIA's PILOTNet) focuses predominantly on unstructured environments using raw, ego-centric camera feeds. Because these systems must implicitly learn to interpret depth, perspective, and 3D geometry from raw pixels, they require deep, computationally expensive Convolutional Neural Networks (CNNs). This requires high-performance onboard hardware (GPUs), which significantly increases system cost and complexity.

There is a distinct lack of research that leverages geometric constraints to simplify

this learning process. Most existing studies either use fiducial markers (like ArUco) solely for coordinate-based localization or rely on heavy deep learning models for raw visual perception. Few have explored bridging these domains by using markers to pre-process the visual feed.

This study addresses this gap by proposing a lightweight architecture where ArUco-based homography is used to transform raw video frames into rectified, top-down (Birds-Eye-View) images. By feeding these geometrically simplified images, rather than raw perspective views, into the neural network, the complexity of the learning task is greatly reduced. This allows a standard CNN to map visual inputs directly to steering angles with high accuracy, enabling robust end-to-end autonomous navigation on significantly constrained hardware.

1.2 | Statement of the Problem

The existing self-driving implementations require cameras and other sensors attached on the moving robot for the real-time view of the navigation path, similar to the point-of-view of a vehicle's human driver. This design is very suitable to the environment where the road or path is highly recognizable and distinguishable from other objects. A solution must be explored to design and implement an autonomous robot for potential use in lawn mower machines or vacuum cleaners where the visible ground path for navigation is not available.

This limitation is compounded by the broader architectural constraints of modern autonomous navigation systems. Prevailing solutions such as Simultaneous Localization and Mapping (SLAM) rely on expensive sensor technology — including LiDAR, depth cameras, and inertial measurement units that are usually paired with computationally intensive software pipelines that demand high-performance onboard processors. While effective in structured and feature-rich environments, these systems are fundamentally mismatched with the operational realities of compact, consumer-grade platforms such as robotic vacuum cleaners and autonomous lawn mowers, which are subjected to severe limitations in unit cost, power consumption, and onboard processing capacity.

Moreover, the absence of a distinguishable physical ground path introduces a deeper challenge beyond mere sensor selection: it invalidates the core assumption upon which most ego-centric, vision-based navigation models are built. These models are

trained to detect and follow physical path features such as lane markings, boundary edges, or color contrast between the drivable surface and its surroundings. On a featureless terrain such as carpet, grass, or bare flooring, these features are entirely absent, making conventional end-to-end visual navigation approaches inapplicable without significant architectural redesign.

This study addresses this gap by proposing an alternative navigation framework that separates path perception from the robot's onboard ego-centric view. By offloading visual processing to a fixed, overhead camera and employing planar fiducial markers as geometric reference anchors, the system grounds navigation in top-down spatial reasoning rather than ground-level path recognition. This approach eliminates the dependency on a visible physical floor path while remaining deployable on resource-constrained hardware, establishing a foundational proof-of-concept for autonomous navigation in environments where the ground surface provides no inherent visual guidance.

1.3 | Research Objective

The following are the general objective and specific objectives of the study.

1.3.1 | General Objective

To develop an autonomous vehicle system with external top-down camera and utilizing an end-to-end Convolutional Neural Network (CNN) architecture that maps raw visual input directly to robot's steering and throttle to navigate a virtual ground path.

1.3.2 | Specific Objectives

1. To implement a robust Inverse Perspective Mapping (IPM) pipeline using ArUco markers for dynamic homography calibration, capable of transforming angled, ego-centric RGB camera frames into rectified, top-down, bird's-eye view (BEV) representations of the local path.

2. To construct a specialized behavioral cloning dataset by manually driving the robot, consisting of synchronized pairs of preprocessed BEV path images

and human-demonstrated steering angles, collected across various track configurations.

3. To train the robot on a regression-based Convolutional Neural Network (CNN) architecture that maps high-dimensional BEV visual inputs directly to continuous steering commands (and optionally linear velocity), utilizing Mean Squared Error (MSE) loss for optimization.

4. To deploy the trained model in the driving pipeline and its inference output is sent directly to the robot to control the movement and drive autonomously

5. To evaluate the autonomous robot by measuring lane-keeping accuracy (CTE/RMSE), heading error, trial-to-trial consistency, and steering actuation behaviour (raw command magnitudes after scaling by k_{steer}), and summarizing results using confusion matrices and per-scenario statistics.

1.4 | Scope and Limitations of the Research

This study focuses on the development of a low-cost, end-to-end autonomous navigation system using a single onboard RGB camera. The system utilizes Inverse Perspective Mapping (IPM) facilitated by ArUco markers to process visual inputs.

The hardware implementation is restricted to computationally constrained platforms (e.g., Raspberry Pi or NVIDIA Jetson Nano) to validate the lightweight nature of the proposed architecture. The robot operates exclusively in structured environments where ArUco markers can be reliably detected, and no 3-D mapping, LiDAR, or depth-based perception is included. Unlike traditional SLAM systems that integrate probabilistic mapping and multi-sensor fusion Cadena et al. (2016); Cieslewski et al. (2017), this study does not attempt full environment reconstruction or dynamic obstacle mapping. Instead, localization is constrained to planar marker fields consistent with research demonstrating accurate 2-D pose estimation using fiducials Garrido-Jurado et al. (2014); Romero-Ramírez et al. (2018).

As for the environmental scope of this study, the navigation environment is limited to structured, planar indoor surfaces with consistent artificial lighting. The robot operates within an area of interest, which is a predefined 2-D plane where ArUco markers are visible and explicitly placed to define track boundaries or reference points.

The system is not designed for outdoor environments, uneven terrain, or varying illumination conditions.

Neural-network steering prediction is trained only on data collected within the experimental environment. Thus, the learned model is not expected to generalize to unstructured or markerless environments, consistent with known limitations of end-to-end visual navigation models regarding generalization and noise sensitivity Bansal et al. (2020); Codevilla et al. (2019). The evaluation metrics, cross-track accuracy, heading error, and success rate, are limited to controlled indoor trials and do not include long-term autonomy, multi-robot coordination, or obstacle-rich navigation.

1.5 | Significance of the Research

This study contributes to the growing demand for accessible and low cost autonomous driving robot navigation frameworks by presenting a system that removes the need for expensive sensors, advanced SLAM algorithms, and high-end computing hardware. While previous works have highlighted that fiducial markers such as Aruco provide efficient localization Garrido-Jurado et al. (2014); Romero-Ramírez et al. (2018), relatively few studies have used these markers into a complete low cost 2-D navigation system suited for educational or research purposes.

This research contributes to the field of robotic control by exploring the potential of End-to-End Imitation Learning in constrained environments. While previous works have validated ArUco markers for localization Romero-Ramírez et al. (2018) and CNNs for visual navigation separately, this study bridges these domains. It investigates whether a homography-based pre-processing step can simplify visual inputs enough to allow a lightweight neural network to provide navigation controls accurately without the computational overhead of deeper, complex architectures.

The study is locally and globally significant because it provides:

1. This study provides a framework for implementing these advanced Deep Learning concepts using only a standard webcam and a microcontroller, making the technology accessible for educational and low-resource industrial applications.
2. A simple experimental platform for teaching concepts related to localization, control systems, and machine learning.

3. A replicable and scalable framework for researchers exploring low-cost autonomous navigation in structured indoor spaces.

Overall, this study aims to address the gaps in the literature by producing a fully integrated, low-cost navigation framework that combines fiducial-marker localization and End-to-End robot control, offering a practical solution for accessible autonomous robot navigation systems.

2

Literature Review

This chapter reviews the existing literature on autonomous mobile recognition systems, with a focus on sensor technologies, marker based localization, and robot control strategies. It analyzes the advantages and limitations of current methodologies to establish the theoretical framework for the proposed low-cost navigation system. Additionally, the Preferred Reporting Items for Systematic Reviews and Meta-Analysis (PRISMA) guidelines was done to ensure a systematic and transparent process of searching, screening, and inclusion of literature.

2.1 | Search Strategy

To begin the literature review process and identify relevant studies, the researchers conducted a literature search across Google Scholar, IEEE Xplore, and arXiv. The search was done specifically with the query strings shown in Table 2.1 below. Query strings were varied for each database to ensure that the strings were syntactically correct with each database's unique search syntax.

2.2 | Inclusion and Exclusion Criteria

The search process yielded a total of 2050 papers but narrowed them down to 15 papers by selecting them based on the inclusion and exclusion criteria shown in Table 2.2.

The papers from all databases were pooled together, and the screening process started. Titles and abstracts were screened to identify studies relevant to the research topic.

DATABASE	Query String
Google Scholar	("ground robot" OR "mobile robot" OR UGV OR "autonomous robot") "vision-based localization" OR "fiducial marker" OR "ArUco" OR "AprilTag" "path planning" OR "trajectory tracking" "neural network" OR "pure pursuit" OR PID
IEEE Xplore	("ground robot" OR "mobile robot" OR "autonomous robot") AND ("vision-based localization" OR "computer vision" OR "fiducial marker" OR "ArUco" OR "AprilTag") AND ("pose estimation" OR homography) AND ("path planning" OR "coverage path planning" OR "trajectory tracking") AND ("pure pursuit" OR PID OR "neural network" OR "learning-based control")
ScienceDirect	("ArUco" OR "AprilTag" OR "fiducial marker") AND ("pose estimation" OR SLAM) AND ("mobile robot" OR robotics) AND (camera OR vision)

Table 2.1: Databases and query strings used in the literature search.

Inclusion	Exclusion
■ Papers published between 2020 and 2025. ■ No paywall. ■ Uses fiducial markers or overhead-based camera localization. ■ Includes mobile robots. ■ Uses Neural Network or Hybrid (Classical + NN) Frameworks or purely classical framework.	■ Studies older than 6 years. ■ Papers with paywall. ■ Studies without using fiducial markers or any camera vision. ■ Studies focusing only on SLAM without fiducial markers, overhead vision, or robot control relevance. ■ Non-ground robots unless used for overhead vision.

Table 2.2: Inclusion and exclusion criteria for literature selection.

2.3 | Results

The systematic review and screening process identified a total of 58 relevant papers after gathering results from all of the databases. All studies were then assessed for eligibility based on the inclusion and exclusion criteria.

During the abstract and title screening, studies that did not use ground mobile robots, fiducial markers, overhead camera, or were otherwise irrelevant to autonomous navigation were excluded. At the full-text assessment stage, a total of 43 studies were excluded due to the following reasons:

- No use of fiducial markers or overhead camera
- UAV-only studies where the primary subject was aerial
- Surveys, reviews, datasets, or patents with no experimental results
- Studies without relevant navigation or control frameworks

After the full-text assessment,a final number of 15 papers were included in the qualitative review. These studies used ground mobile robots, fiducial markers, overhead camera-based localization, and also added a neural, hybrid, or purely classical control frameworks for navigation. A prisma flow diagram summarizing this process is presented in Figure 2.1below. Lastly, in addition to the studies included in our systematic review(PRISMA), other relevant literature, especially the ones provided by the adviser of this research, was also considered to provide context and background for this study.

2.1

2.4 | Autonomous Navigation and Sensor Technologies

Autonomous navigation fundamentally relies on a mobile robot's ability to perceive its environment and determine its position relative to the environment. While autonomous navigation is a well established field, the hardware requirements for reliable performance remain a significant barrier for academic and low cost applications. In our review of literature on autonomous navigation, we found out that modern autonomous navigation relies on a heterogeneous sensor suite. Some of the most widely adopted technologies range from the classic RGB-D cameras , to innovations like event based sensors (Gatesichapakorn et al., 2019; Rodríguez-Martínez et al., 2025). Recent multi-sensor approaches such as the work of Dai and Lee (2023), who integrated LiDAR, YOLO-based AprilTag detection, and depth cues for

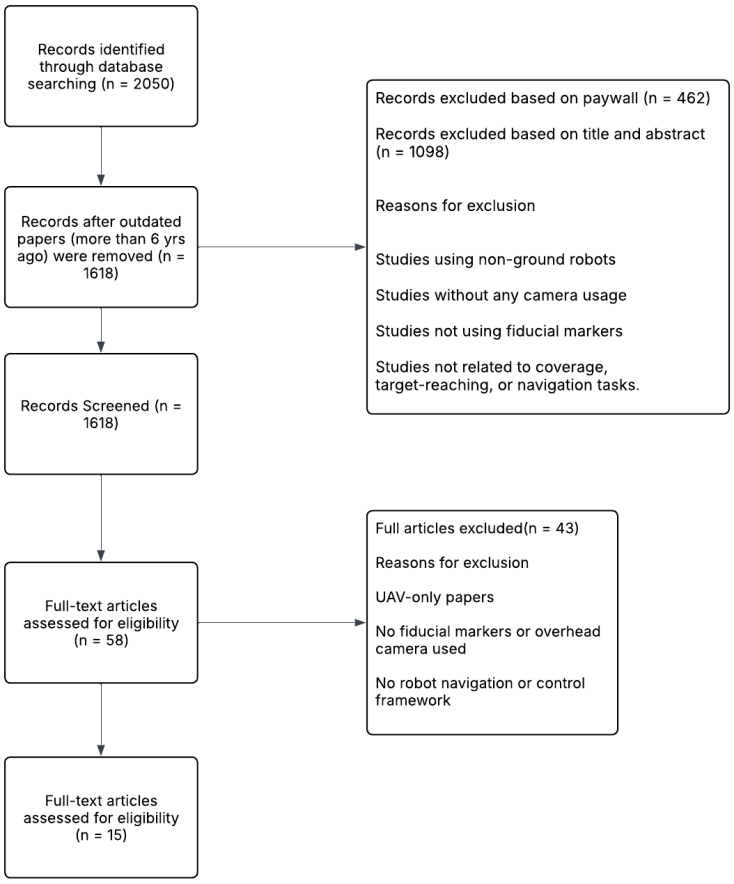


Figure 2.1: PRISMA 2020 Flowchart

autonomous docking, demonstrate how heterogeneous sensor fusion significantly improves docking precision and reliability in cluttered indoor environments. In this subchapter, we evaluate the advantages and disadvantages of each sensor technology.

2.4.1 | Monocular and Multicamera RGB Systems

Monocular RGB cameras remain one of the most widely used sensing modalities in autonomous navigation due to their low cost, size, and ease of integration. Recent work demonstrates that deep learning greatly improves the capability of monocular systems, enabling better perception even without clear depth sensors. Research has shown that convolutional and transformer-based networks can infer scene geometry, obstacle layout, and traversability directly from RGB images (Sleaman et al., 2022). Additional

monocular systems, such as the neural-network-driven visual servoing framework by Jokić et al. (2019), show how wide-FOV fisheye lenses combined with CNN-based controllers can enhance robustness even when depth cues are limited. Similarly, Ho and Lin (2020) demonstrated that pose-based visual servoing enhanced with lightweight deep-learning binarization improves tracking accuracy in resource-limited mobile robot platforms. These methods reduce hardware complexity while maintaining acceptable localization performance for indoor and structured environments. Despite these advantages, monocular cameras suffer from an inherent depth ambiguity that can limit reliability in dynamic or cluttered spaces. Studies on monocular systems trained with deep reinforcement learning highlight that navigation performance largely depends on the variety of training environments (Yokoyama and Morioka, 2020). To help solve the depth limitation, recent research combines multiple deep learning modules and depth prediction to create better feature representations for navigation, as shown by Machkour et al. (2022). These hybrid pipelines increase robustness but require more computational resources than classical monocular methods.

2.4.2 | RGB-D Cameras

While Monocular and multi-camera RGB systems provide useful visual data for navigation, their lack of direct depth information restricts their robustness, especially in tasks requiring reliable 3D localization or obstacle avoidance. RGB-D cameras solve this problem by combining regular RGB images with depth information, producing a richer representation that simplifies tasks such as pose estimation, mapping, and collision detection. According to Civera and Lee (2017), RGB-D sensors are good alternatives to traditional range sensors like LiDAR because they are often cheap, compact, and require low-power, yet still provide both color and depth data that support 3D scene reconstruction. RGB-D sensing also appears in navigation pipelines such as the omnidirectional manufacturing-oriented robot proposed by Patruno et al. (2020), where combined RGB-D perception enables reliable autonomous movement in industrial indoor environments.

Many state-of-the-art approaches combine RGB-D data with odometry / SLAM pipelines to exploit this richer input. For instance, Gatesichapakorn et al. (2019) demonstrated a practical indoor navigation system using fused inputs from a 2D LiDAR and an RGB-D camera. This hybrid setup allowed a real mobile robot to navigate without a precomputed map. On the other hand, recent work such as Kostusiak and Skrzypczynski (2023) shows how modern deep-learning techniques can

further improve performance: by refining noisy or incomplete depth maps (common with low-cost Kinect-like sensors), the study achieved more reliable indoor visual odometry under budget-friendly constraints.

2.4.3 | LiDAR and Camera-LiDAR Hybrids

Recent studies highlight that combining LiDAR with cameras significantly improves autonomous navigation systems. According to Kolar et al. (2020), combining data from LiDAR and cameras lead to more accurate and robust mapping and localization, than just relying on either modality alone. There are studies that support this view, for example, De Silva et al. (2018), designed a framework that combined the use of LiDAR and wide-angle camera images and it resulted in significantly improved free-space detection performance. Hybrid LiDAR-vision architectures further align with the findings of Liu et al. (2022), who showed that fusing CNN-based visual relocalization with progressive LiDAR scan-matching significantly enhances robustness under viewpoint changes and degraded visual conditions. However, these papers also highlight the limitation of the LiDAR-camera fusion which requires precise calibration and consistent environmental conditions that make real-time deployment more computationally demanding.

2.4.4 | Event-Based Cameras

Event-based cameras output streams of "events" rather than fixed video frames, because of this, they offer compelling advantages for robotics and SLAM. According to Tenzin et al. (2024), event cameras deliver high temporal resolution, low latency, reduced power consumption and data bandwidth compared to traditional frame-based cameras. Additionally, the hardware architecture of this type of camera can efficiently handle the asynchronous event streams which enable low-power SLAM and perception on embedded robotic systems.

2.4.5 | Infrared and Thermal Imaging Sensors

Infrared and thermal imaging sensors give strong advantages for autonomous robots especially in visually degraded and low-light environments. Papachristos et al. (2018) showed that thermal imaging provides reliable localization for robots even in smoke, dust, or foggy conditions where performance of other cameras like RGB and LiDAR deteriorates. Similarly, recent work on Thermal SLAM by Salem and Gedik (2025).

demonstrates that temperature-based features can be used for object detection and tracking which improves localization regardless of lighting.

2.4.6 | Comparative Summary

Sensing Modality	Key Advantages	Major Limitations
Monocular and Multicamera RGB	■ Low cost, compact size, and ease of integration. ■ Simplifies mapping and collision detection.	■ Prone to noise or incomplete depth maps. ■ Often limited to indoor ranges.
LiDAR & Camera Hybrids	■ Significantly improved free-space detection. ■ More accurate and robust mapping than the single standalone of each part.	■ Requires precise calibration. ■ Computationally demanding for real-time deployment. ■ Requires environmental consistency.
Event-Based Cameras	■ High temporal resolution and low latency. ■ Reduced power consumption and data bandwidth.	■ Requires specific hardware architectures. ■ Output is non-traditional (events) which requires specific processing.
Infrared & Thermal Sensors	■ Reliability in visually degraded conditions. ■ Effective even in low-light, low visibility environments.	■ Primarily focused on visually degraded conditions. ■ Less feature rich for standard navigation.

Table 2.3: Sensing modalities, advantages, and limitations.

As shown in Table 2.3, while LiDAR and Hybrid systems offer high accuracy, they suffer from high computational demands and calibration complexity. Conversely, Monocular RGB systems (Section 2.2.1) offer a balance of low cost and hardware simplicity. By leveraging Deep Learning—specifically the Neural Network architecture inherent in the DonkeyCar framework—this study aims to mitigate the 'depth ambiguity' limitation of monocular cameras by supplementing local vision with a global overhead supervisor (ArUco), thereby maintaining the low-resource benefits of RGB systems while improving navigation reliability.

2.5 | Computer Vision for Autonomous Robot Systems

The computer vision systems in robotics systems serve as the eyes of the robot – providing vision that helps it recognize and understand its surroundings. Studies such as Alimovski et al. (2022) also show that camera-only visual localization pipelines can remain effective even in complex, GPS-limited outdoor urban environments. Computer vision as defined by Vina (2023), is the technique where robotic systems use cameras and sensors to gather visual information.Then the information gathered is then processed by algorithms, which enable robots to perform complex tasks like assembly, quality control, and navigation.

2.5.1 | LiDar and RTK-GPS and its Limitations

LiDar and RTK-GPS (Real-Time Kinematic GPS), alone or in combination, are widely used to produce accurate mapping for vehicles (de Miguel et al., 2020; Thepsit et al., 2023). However, there are several limitations that push for the implementation of alternative approaches for autonomous systems.

2.5.1.1 | Cost, Hardware, and Computational Requirements

LiDar systems, as described by Whitaker et al. (2020), are often expensive and sizable for some applications. Lu et al. (2021) supports this idea in their study, concluding that LiDar is the most promising method for producing accurate, real-time localization among other approaches but suffers from the limitations of the system's computational ability, requiring more powerful CPU/GPU. This limitation poses a problem for smaller systems such as mobile robots where money, weight, and power consumption directly impact the navigation operations.

2.5.1.2 | GPS Signal Degradation and Denial

RTK-GPS systems may suffer from extreme performance degradation or complete signal loss in a constrained environment such as urban canyons, underground or multi-level parking structures, tunnels, and indoor spaces (Bhat and Kavasseri, 2024; Fang et al., 2020; Jeon et al., 2021). Odometry errors accumulate fast without external correction, leading to inaccuracy in position estimates. Massa et al. (2020) resorted to alternative methods for a localization of self-driving racecars because GPS can be unreliable on racing tracks and in harsh environments.

The limitations for LiDar and GPS collectively motivate the exploration of modern sensing modalities that can provide reliable localization in challenging scenarios where traditional systems struggle (Dreissig et al., 2023; Zhang et al., 2023). This necessitates a need for developing alternative low-cost computer-vision approaches based on artificial fiducial markers that can provide reliable positioning without expensive sensors.

2.5.2 | Fiducial Marker Methods

Fiducial markers are designed visual patterns which are machine or camera-detectable strategically placed within an environment to provide reliable localization mapping or pose estimation cues for autonomous systems. These visual markers serve as the landmarks that may offer multiple advantages over resource-intensive, natural or feature-based localization techniques. A broader mapping of marker-based navigation techniques is summarized in the systematic review by Alghamdi et al. (2025), which highlights the lightweight computational requirements, reliability, and adaptability of fiducial marker systems across diverse robotic domains.

In comparative studies of standard, custom, reflective, and infrared markers, fiducial markers generally offer high detection accuracy, robustness, and low computational cost, but their performance can still degrade under poor lighting conditions Alghamdi et al. (2025). Standard physical markers will include but not limited to, AR Tag (Fiala, 2005), AprilTag (Olson, 2011), and ArUco markers (Garrido-Jurado et al., 2014).

2.5.2.1 | Operational Advantages

When compared to localization methods like GPS RTK, the pragmatic advantages behind physical fiducial markers come from their ability to provide discrete, absolute position fixes that can correct accumulated drift in dead-reckoning systems (Park and Cho, 2023; Xu et al., 2023)

Markers also facilitate the creation of persistent maps with known landmark positions. Unlike natural feature-based SLAM, which must continuously track and update potentially ambiguous environmental features, marker-based systems can rely on the stable, unambiguous identity and position of installed markers. This stability simplifies map maintenance and enables more predictable localization performance (Muñoz-Salinas et al., 2018; Pfrommer and Daniilidis, 2019; Wang et al., 2023).

2.5.2.2 | Applications in Autonomous Systems

Fiducial markers were leveraged to localization for vehicle parking in a GPS-denied setting like an underground parking lot for precise autonomous driving (Fang et al., 2020). Moreover, indoor autonomous drones use ceiling or wall mounted markers for positioning during warehouse inventory and structural examination tasks (Nahangi et al., 2018). Marker systems have also been applied in assistive indoor navigation, such as the ARUCO-based pan-tilt camera system demonstrated by Adapa et al. (2019) for improving mobility support for blind and visually impaired users. Overhead or airborne localization frameworks similarly leverage markers, as seen in Xu et al. (2023), who demonstrated a UAV-based visual navigation method using squared planar fiducials for orientation and path-following.

These applications share a common theme: they operate in environments where GPS is unavailable or unreliable, infrastructure modification is feasible, and cost-effective localization solutions are essential. Fiducial markers address these themes by providing a pragmatic middle ground between expensive LiDAR systems and drift-prone dead-reckoning approaches. The following section (2.5.3) details how these fiducial marker principles are realized in marker-based localization and mapping architectures, with a focus on ArUco-based systems.

2.5.3 | Marker-Based Localization, Mapping, and ArUco Systems

Building on the general properties of fiducial markers outlined in Section 2.2.2, this section examines marker-based localization and mapping pipelines, emphasizing ArUco-based implementations in autonomous vehicles and robots.

2.5.3.1 | Detection and Pose Estimation

Planar fiducial marker systems usually follow a structured pipeline that begins with image acquisition and proceeds through marker detection, corner localization, and pose estimation. In common implementations such as ArUco, candidate markers are extracted by thresholding and contour detection, followed by decoding of the internal binary pattern to recover marker identity and reject false positives (Rosebrock, 2024). Once the four marker corners are localized in image coordinates and associated with a known marker size, the perspective-n-point (PnP) algorithm is used together with the camera calibration parameters to obtain the 6-degrees-of-freedom (6-DoF) transformation between the marker frame and the camera frame (Braun et al., 2022; Fang et al., 2020). This geometric formulation avoids any need for training data, and it provides an interpretable pose estimate that can be directly integrated into downstream localization and mapping pipelines.

2.5.3.2 | Robot and Camera Pose Tracking with ArUco

Real-time implementations such as the UAV-mounted camera system in Waqas and Cha (2022) confirm that marker-based pose tracking remains feasible even under dynamic motion, non-static camera poses, and rapid changes in scene geometry. Additionally, lightweight educational platforms, such as those in the Duckietown initiative by Krinkin et al. (2019), highlight how careful camera calibration and wheel-encoder integration support accurate pose estimation even on low-cost robotic systems. When fiducial markers are deployed in the environment, each ArUco detection yields a relative pose measurement between the camera and the corresponding marker. Assuming a calibrated camera and known marker dimensions, these measurements can be expressed in metric units and transformed into a robot-centric or world-centric frame, enabling continuous tracking of the robot's position and orientation as it moves through the environment (Montecchio, 2022; Zhou et al., 2022). In practice, the system accumulates a time series of marker-based pose updates, which are combined with kinematic information such as wheel odometry to produce a smooth estimate of the robot trajectory in GPS-denied settings.

2.5.3.3 | ArUco-based Planar Task-Space Localization

Aside from providing global or relative pose measurements, planar fiducial configurations can be used to define local task spaces through homography estimation (Braun et al., 2022). Numerous experimental systems have demonstrated the practicality of ArUco markers for autonomous vehicle and robot localization.

By placing multiple ArUco markers at the corners of a region of interest, Huang et al. (2022) explored an optimized system that can compute a projective transformation between image coordinates and a planar world coordinate frame aligned with the physical surface. In addition, Rosner et al. (2025) follows this idea for indoor drone tracking through markers on walls that define a room coordinate system (inside-out tracking). This demonstrates the efficacy of fiducial markers to provide planar task-space localization. Oh and Kim (2025) applied this system for docking stations to achieve robust positioning with low-cost cameras and embedded processors.

2.5.3.4 | Sensor Fusion Architectures

In conventional autonomous systems, marker-based pose estimates are typically fused with proprioceptive sensors to compensate for intermittent visibility and measurement noise (Kumar and Muhammad, 2023). Extended Kalman filters, error-state Kalman filters, and particle filters are widely used to combine discrete, absolute pose fixes from fiducial markers with continuous estimates derived from wheel encoders and inertial measurement units (De La Puente et al., 2025; Ullah et al., 2020). Marker–odometry fusion has also been demonstrated effectively by Alam et al. (2023), who showed that integrating AprilTag observations with particle filtering reduces drift and improves localization accuracy in indoor autonomous robots. In systems like these, the prediction step propagates the robot state forward using odometry or motion models, while the update step incorporates new marker observations when available, bounding drift and improving robustness when there is sensor noise and environmental disturbance.

2.5.3.5 | Marker-aided SLAM and Mapping

In recent meta, mapping for autonomous robots are commonly carried out through LiDAR vision, which is costly and complex when it comes to maintenance Chen et al. (2025). Traditional Marker-aided SLAM formulations incorporate fiducial markers as explicit landmarks in the map, rather than relying solely on natural point or line features Tourani et al. (2023). In graph-based SLAM, markers are represented as

nodes with fixed or estimated poses, and edges encode relative constraints between robot poses and marker observations; optimization of this graph improves both robot trajectories and marker maps Muñoz-Salinas et al. (2018). This approach simplifies data association, because each marker carries a unique identifier, and supports persistent, globally consistent maps that can be reused across sessions, which is particularly advantageous in structured environments such as parking facilities and industrial halls.

Recent implementations reinforce the practical value of this paradigm. For example, Tolenov and Omarov (2024) demonstrated a real-time camera-based SLAM pipeline capable of mapping unknown indoor environments without the use of LiDAR, showing that lightweight vision systems can be competitive when supported by robust tracking and optimization. Marker-enhanced localization pipelines also appear in aerial–ground hybrid systems such as Asif et al. (2020), where a tethered UAV and ground robot perform collaborative localization using deep neural networks supplemented by intermittent fiducial marker detections. These studies highlight how marker-aided SLAM can reduce cost, simplify data association through unique IDs, and enable persistent, reusable maps which are particularly beneficial in structured indoor environments.

2.5.3.6 | Gap Analysis for Marker-Based Detection and Localization

While the literature establishes that marker-based sensor fusion and SLAM architectures (discussed in Sections 2.5.3.4 and 2.5.3.5) provide robust state estimation, they inherently introduce significant computational complexity and latency due to the iterative nature of Kalman filtering and graph optimization. Moreover, these traditional pipelines rely heavily on explicit state estimation. This requires the system to calculate the exact coordinates at every time step before making control decisions. This dependency introduces architectural complexity and computational overhead. Section 2.6 reviews recent advancements in deep learning for robot control, providing further justification for the shift away from traditional pose estimation toward end-to-end architectures.

2.5.4 | Modern Detection and Real-Time Systems

Autonomous robots require vision-based localization pipelines that operate under strict real-time and resource constraints, often on embedded platforms such as single-board computers or low-power system-on-chip devices. The detection and pose estimation of

fiducial markers must run at sufficient frame rates to support closed-loop control, while sharing computational resources with other perception and control modules. This has motivated a line of work focused on optimizing fiducial detection algorithms, memory usage, and numerical routines so that marker-based localization remains feasible on platforms with limited CPU, GPU, and power budgets Yulianto et al. (2025).

2.6 | Deep Learning Approaches for Robot Control Strategies

The integration of neural networks in autonomous systems have shifted the industry standard from hand-coded features, to data driven methodologies. In the context of autonomous mobile robot navigation, deep learning techniques have been increasingly used to process complex sensory data and generate navigational decisions. Upon reviewing literature, a survey from Xiao et al. (2022) shows three deep learning paradigms for robot navigation; End-to-End Learning (E2E), Learning Subsystems, and Learning Components. Additionally, specific neural network architectures and training methodologies, referred to here as deep learning techniques, are employed across these paradigms to handle varied input modalities and control objectives. Recent marker- and vision-based navigation studies reinforce this trend. For instance, Liu et al. (2022) demonstrated that CNN-based relocalization can significantly improve pose recovery within a hybrid LiDAR-vision localization framework, showing how learned perception compensates for degraded geometric features. Deep neural controllers have also been applied to monocular systems, such as the visual servoing approach by Jokić et al. (2019), which uses fisheye camera imagery to control a wheeled mobile robot. Similarly, Ho and Lin (2020) leveraged lightweight deep-learning binarization to enhance pose-based visual servoing for resource-constrained mobile robots. Hybrid aerial-ground systems such as Asif et al. (2020) also illustrate how DNNs can infer relative positions between robots when fiducial marker signals are intermittent or partially occluded. Together, these systems demonstrate that deep learning serves not only as a replacement for classical control but also as a complementary module for perception, relocalization, and decision-making within modern autonomous navigation architectures.

2.6.1 | Deep Learning Scopes

The application of deep learning in autonomous mobile robot navigation is categorized by how much of the classical navigation stack (perception, localization, planning, and control) is replaced by neural networks. If the whole system is replaced with a neural network (E2E), if only subsystems of the navigation system are replaced (Learning Subsystems), or if only components (Learning Components).

End-to-End (E2E) Learning seeks to replace the classical navigation stack with a single deep neural network that maps raw inputs directly to control actions. This approach centralizes the process by integrating it into a single neural network process. Kim et al. (2018) demonstrated this by developing a deep learning model that processes RGB camera data to output steering in a single step. Similarly, Pierre (2018) applied E2E learning to robotic following behaviors, proving that a single network could learn complex tracking tasks without explicit object recognition modules.

Deep Learning Components, unlike E2E, involve replacing a single processing step within a module with a deep learning model. The most common application is in the perception module, where Convolutional Neural Networks (CNNs) replace manual feature extractors to perform object detection or semantic segmentation. For example, Redmon et al. (2016) introduced YOLO, a real-time object detection system that serves as a perception component, feeding bounding box coordinates to a classical costmap or planner. In this scope, the neural network does not make navigational decisions; it strictly provides structured information to a classical algorithm.

Deep Learning Subsystems replace an entire subprocess in the multi-step process of the autonomous robot navigation stack, while retaining the classical methods for the other parts. One such example is Mohanty et al. (2016) DeepVO architecture proposal. The deep learning architecture replaces the classical visual odometry system with a Recurrent Convolutional Neural Network (RCNN). This subsystem takes raw video as input and outputs the robot's pose, directly replacing the localization module while leaving the planning and control modules untouched. A similar study from Faust et al. (2017) proposed "PRM-RL," which replaces the local path planner subsystem with a Reinforcement Learning agent to handle dynamic obstacle avoidance, while still relying on a classical Probabilistic Roadmap (PRM) for global path planning.

2.6.2 | Deep Learning Techniques

Navigation systems often rely on specific neural network architectures and learning algorithms to process distinct data inputs, and generate control policies as outputs. Upon critical review on Literature, it is evident that the deep learning approaches of autonomous mobile robot navigation are driven by three architectural paradigms: Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Vision Transformers (ViTs).

Convolutional Neural Networks are the most widely used approach in these systems. This can be attributed to the fact that CNNs are the best fit for the navigation pipeline. CNNs are best suited for processing spatial data such as camera images, depth maps, and lidar projections. The CNN can extract localized information from edges and textures to semantic level concepts, allowing robots to interpret complex environments reliably and efficiently (LeCun et al., 2015). This makes CNNs the best fit for perception tasks such as obstacle detection (Eigen et al., 2014; Redmon et al., 2016; Ronneberger et al., 2015). This makes it the best fit for replacing the classical localization part of the navigation stack.

Recurrent neural networks (RNNs) are widely used in robot navigation due to their ability to model dependencies in sequential data such as robot odometry, velocity history, etc. Unlike CNNs, RNNs with LSTMs, and or GRUs are essential for predicting sequential robot trajectories, estimating future states, and generating smooth control commands (Cho et al., 2014; Hochreiter and Schmidhuber, 1997). Recent Research incorporates RNNs with reinforcement learning or hybrid deep learning pipelines to better model robot-to-environment interactions. One example is the hybrid RNN architecture proposed by Radha and Karthikeyan (2025) which demonstrates an improvement on optimization of path planning and collision avoidance by fusing temporal motion cues with spatial features extracted from CNNs Although studies also show that that RNN-based models significantly improve localization accuracy by including historical context, reducing hallucinations and localization drift, denoising movements, specifically in environments where GPS is not available (Ichter et al., 2018; Weng et al., 2019).

Vision Transformers (ViTs) are a newer deep learning technique that is used in robot navigation, leveraging the breakthroughs in self-attention and the transformer model for vision based tasks. Unlike CNNs, which look at a group of fields at a time, CNNs process the data, its dependencies, and the semantics of each datapoint relative to

each other. This helps robots interpret the whole environment, and consequently, plan out safe paths more effectively. (Dosovitskiy et al., 2021; Khan et al., 2022). Recent studies show that ViTs outperform CNNs in tasks requiring higher level semantic understanding or contextual awareness, such as large scale mapping, multiple map localization, contextual path planning, etc. (Chen et al., 2021). Their ability to maintain performance under viewpoint changes, lighting variations, and occlusion makes them increasingly relevant for robust real-world navigation.

2.7 | Theoretical Framework

This section outlines the mathematical models and fundamental theories utilized in the development of the autonomous navigation system. Specifically, it discusses the geometric principles of homography for localization, the algorithmic basis of fiducial marker detection, and the architecture of the deep learning approach used in the study.

2.7.1 | Projective Geometry and Planar Homography

Since this study uses a single overhead camera to track a robot on a flat floor, the relationship between the camera's image plane and the real world ground plane is defined by a planar homography. In computer vision, homography is defined as a fundamental projective transformation that maps points from one planar surface to another Hartley and Zisserman (2003). In our case, given an image point $p = (u, v, 1)^t$ and a corresponding point in the ground-plane coordinate system $p' = (x, y, 1)^t$, the planar homography relates them as

$$s \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

where

- s is an arbitrary scaling factor,
- H is the 3×3 homography matrix,
- (u, v) are the pixel coordinates of the marker,

- and (x, y) are the metric coordinates on the floor.

To solve for the matrix H, at least four point correspondences are required. This study leverages the Direct Linear Transformation (DLT) algorithm to compute H using calibrated anchors at the corners of the navigation arena, allowing for the real-time conversion of visual data into metric position estimates (Szeliski, 2010).

2.7.2 | Neural Networks and Convolutional Neural Networks (CNNs)

The control strategy that we proposed in this study is a CNN-based perception to actuation framework to provide end-to-end navigation controls. Unlike classical control methods like PID, which rely on a mathematical method for error correction, this study uses neural networks to approximate the non-linear relationship between the visual input and motor control, effectively emulating the thought process of a human driver. A neural network is a model that aims to imitate the functions of a human brain. It is made up of interconnected nodes (neurons) that are organized into different layers (Goodfellow et al., 2016). The fundamental operation involves transforming input data x into an input y through a series of weighted sums and non-linear activation functions:

$$h_i = f_i(W_i h_{i-1} + b_i)$$

where h_i is the output of the i-th layer, W_i and b_i are the weight matrix and bias vector, and f_i is the nonlinear activation function.

This study employs a Convolutional Neural Network (CNN) architecture, which is highly effective for processing grid-like data such as images (LeCun et al., 2015). CNNs are characterized by three primary layer types that allow the system to learn spatial hierarchies automatically.

Convolutional Layers. These layers apply a set of kernels that slide over the input image, performing convolution operations to extract spatial features such as edges, textures, etc. The operation is defined as:

$$(K * I)(i, j) = \sum_m \sum_n I(i - m,\, j - n)\, K(m, n)$$

where I is the input image, K is the kernel, and (i, j) are the spatial coordinates.

Pooling Layers. These layers downsample the feature maps, reducing the dimensionality and the number of parameters. This helps make the features robust to slight shifts or distortions (e.g., Max-Pooling) while lowering the computational load.

Fully Connected Layers. Typically placed at the end of the CNN, these layers flatten the high-level features extracted by the convolutional and pooling layers and map them to the final output. In this study, the output corresponds to the continuous motor power commands (ie. left and right wheel velocities for Ackermann steering).

In the context of this research, CNN functions as an end-to-end policy network. It takes the homography-transformed overhead image, augmented with a digitally rendered coverage path, as input and regresses the optimal motor controls required to track that path. The network is trained using Behavioral Cloning, where the loss function (e.g., Mean Squared Error) minimizes the difference between the network's predicted actions and the human demonstrator's commands during the data collection phase. This approach allows the system to learn complex path-following behaviors directly from visual data, serving as a learned path follower component that bridges the gap between perception and control.

2.7.3 | Imitation Learning and Behavioral Cloning

The study uses Imitation Learning as its deep learning approach. Imitation learning is the industry standard end-to-end approach based on our literature on Chapter 2. Imitation learning assumes access to expert demonstration. The goal is to learn a control policy that mimics the behavior of the expert given specific state observations (Osa et al., 2018). More specifically, this study implements Behavioral Cloning, the simplest and most direct form of Imitation learning. Using this, the problem of learning a robot's control policy is then reduced to a supervised learning task. The system collects a dataset D consisting of N pairs of observations and expert actions:

$$D = \{(s_i, a_i)\}_{i=1}^{N}$$

where s_i represents the state observation (input) at time step i, and a_i represents the expert action (output) at time step i.

3

Methodology

3.1 | Hardware Setup

3.1.1 | Robot Platform

The mobile agent utilized in this study was a custom-built differential drive robot designed for low-latency execution of remote commands. The platform is intentionally minimized to function as a "remote-controlled" agent, removing the need for heavy onboard computation. Figure 3.1 shows the fully assembled robot platform used throughout this study.

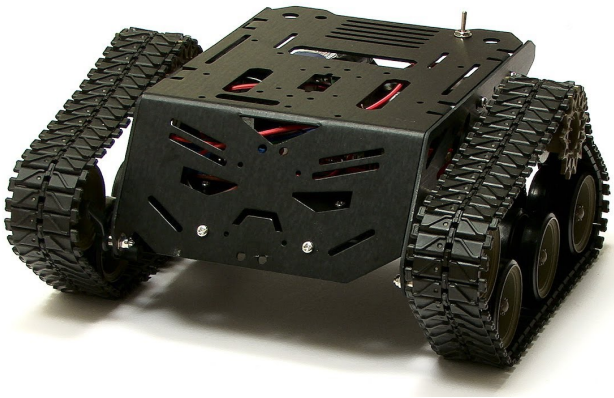


Figure 3.1: Fully assembled custom-built differential drive robot platform, consisting of the DFRobot Devastator chassis, Raspberry Pi 3B, L298N motor controller, and onboard power supply.

3.1.1.1 | Robot Chassis and Drivetrain

The mobile agent utilized a DFRobot Devastator Tank Mobile Platform. This platform operates on a 3–8V DC working voltage and features two micro DC geared motors. It weighs 780 g, measures 8.86 $\times$ 8.86 $\times$ 4.25 inches, and includes driver wheels with a 43mm diameter. The chassis is configured for Differential Drive kinematics, which allows for zero-radius turning and precise heading adjustments. Figure 3.2 shows the DFRobot Devastator Tank Mobile Platform used as the base chassis for this study.

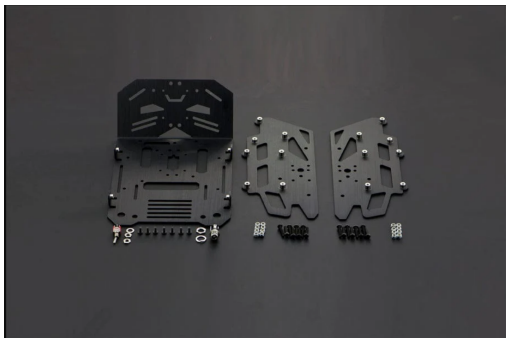


Figure 3.2: DFRobot Devastator Tank Mobile Platform. The chassis supports Differential Drive kinematics via two micro DC geared motors, enabling zero-radius turning and precise directional control.

3.1.1.2 | Embedded Controller

The robot was equipped with a Raspberry Pi 3B running Raspi OS, acting as the low-cost microcontroller and communication bridge. Its sole function was to receive data packets from the central workstation and convert them into Pulse Width Modulation (PWM) signals. Figure 3.3 shows the Raspberry Pi 3B unit used in this study.

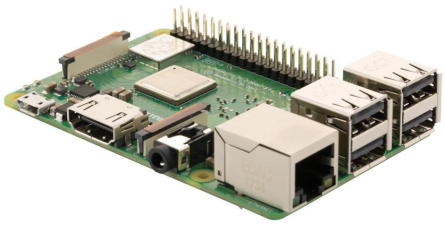


Figure 3.3: Raspberry Pi 3B single-board computer, serving as the onboard microcontroller responsible for receiving control commands from the workstation and generating PWM signals to drive the motors.

3.1.1.3 | Motor Controller

An L298N H-Bridge DC Motor Controller was used to interface the microcontroller with the DC motors. This provides the necessary current amplification and direction control. Figure 3.4 shows the L298N H-Bridge module integrated into the robot platform.

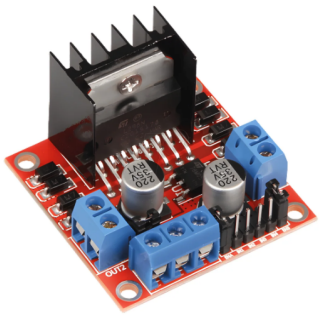


Figure 3.4: L298N H-Bridge DC Motor Controller, responsible for current amplification and bidirectional motor control between the Raspberry Pi and the drive motors.

3.1.1.4 | Wireless Handheld Controller

A Logitech Gamepad F710 Joystick Controller with a USB Dongle was utilized for manually controlling the robot during the data collection phase. Figure 3.5 shows the Logitech F710 controller used during teleoperation sessions.

Figure 3.5: Logitech Gamepad F710 wireless joystick controller used during manual teleoperation for behavioral cloning dataset collection.

Power Supply

The system was powered by a Lithium-Ion battery pack (e.g., 2S or 3S 18650 configuration) to provide stable voltage to the microcontroller and raw power to the motor driver.

3.1.2 | Environment Setup

The experimental environment was a controlled, structured arena designed to implement computer vision processing. A single high-definition RGB webcam (Logitech BRIO) was mounted at a fixed height above the area. The camera was positioned at an angle to cover the entire operational area. While the camera was angled, the software pipeline corrected this perspective distortion mathematically. The arena consisted of a flat, non-reflective surface (to minimize glare and specular highlights that could confuse the CNN). The boundaries of the Region of Interest (ROI) were physically defined by four ArUco markers fixed at the corners. These markers served as the static reference anchors for the system's homography calibration. All image processing, neural network training, and real-time inference were performed on a desktop computer equipped with a GPU. Another laptop workstation ran the Python-based control software, processed the camera feed via USB, and broadcast control commands to the robot over the local wireless network.

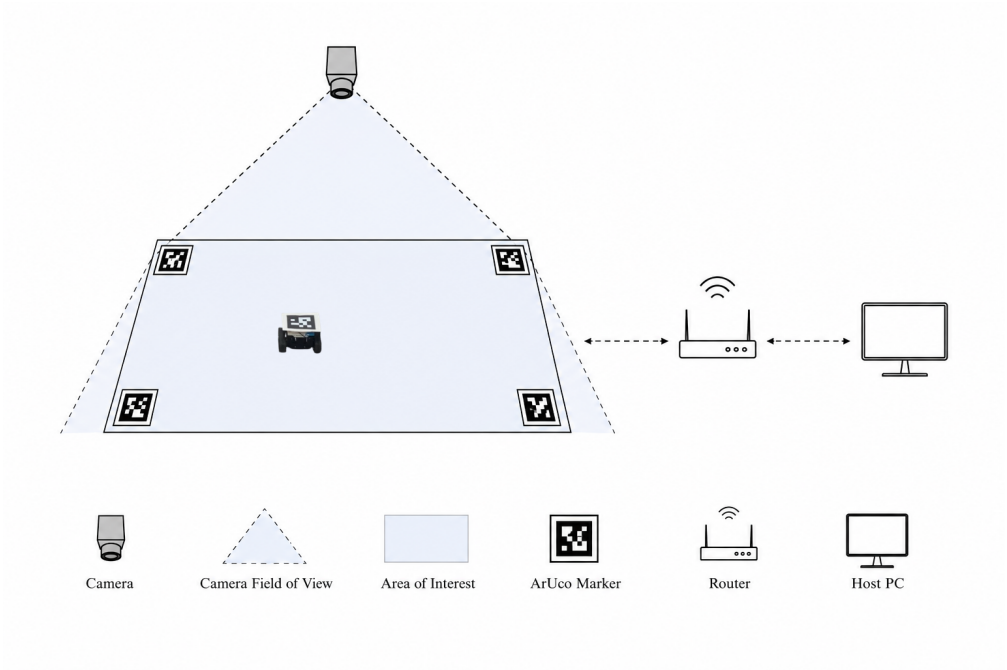


Figure 3.6: Physical environment setup showing the overhead camera, workstation, wireless communication link, and mobile robot within the ArUco-defined arena.

3.1.3 | Training and Inference Hardware

To evaluate the lightweight nature of the proposed end-to-end architecture, the computational workload was divided between a high-performance desktop for offline training and a more constrained mobile system for real-time deployment.

Offline Training Hardware

To support the intensive computational demands of neural network optimization, a dedicated high-performance workstation was utilized. The training system was powered by an Intel Core i9-12900K processor paired with 32 GB of system RAM, operating on Ubuntu Linux. Deep learning operations were offloaded to an NVIDIA GeForce RTX 4060 Ti Graphics Processing Unit (GPU) equipped with 16 GB of VRAM. During the offline Behavioral Cloning phase, this VRAM capacity easily accommodated the approximately 9,000-image dataset and the 2.2-million-parameter modified DAVE-2 architecture, allowing for efficient backpropagation and rapid model convergence.

Real-Time Inference and Processing Hardware

During physical deployment, the role of the central processing hub was transferred to a computationally weaker mobile system to validate the efficiency of the pipeline. Specifically, a Lenovo IdeaPad 1 (14ALC7) laptop running Fedora Linux 42 (Workstation Edition) was utilized. This inference machine was equipped with an AMD Ryzen 5 5500U processor with integrated Radeon Graphics and 16 GB of RAM.Despite lacking a dedicated high-end GPU, this hardware was responsible for executing the synchronous control loop. It successfully handled the OpenCV-based ArUco detection, dynamic homography calculation, egocentric cropping, and the CNN forward passes. The system consistently met the target 20 Hz operating frequency, rapidly mapping the preprocessed 200×200 pixel visual state to continuous steering commands before transmitting the payload to the robot's onboard microcontroller over the local network.

3.2 | Software Framework

3.2.1 | Core Libraries

- **Python**: The system was designed as a modular, monolithic application that was primarily implemented in the Python programming language.
- **OpenCV**: This library was utilized extensively for the computer vision and perception modules. Specific applications of OpenCV in the system included:
  - Estimating the intrinsic camera parameters using standard checkerboard calibration methods (e.g., the `calibrateCamera` function).
  - Extracting, decoding, and filtering ArUco markers, as well as refining corner coordinates to sub-pixel accuracy using the `cornerSubPix` algorithm.
  - Computing the analytic 3×3 transformation matrix using the `getPerspectiveTransform` function and applying the top-down perspective warp to the raw camera feed via the `warpPerspective` function.
- **PyTorch**: This deep learning library was utilized to construct and execute the Inference Engine (the CNN policy). They were responsible for handling the continuous regression tasks of the modified NVIDIA DAVE-2 Convolutional Neural Network architecture.

3.2.2 | Camera and System Calibration

To ensure the visual input accurately represents the physical world, the system underwent a two-step calibration process that separates intrinsic camera properties (lens distortion) from the geometric relationship between the camera and the arena (homography).

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{3.1}$$

where:

- (X, Y, Z) are the coordinates of a 3D point in the world coordinate space.
- (u, v) are the coordinates of the projection point in pixels.

- s is a scaling factor.

In practice, wide-angle lenses such as the Logitech BRIO introduce radial and tangential distortion Brown (1966). These are modelled by the following correction terms:

Radial Distortion:

$$x_{corrected} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \tag{3.2}$$

$$y_{corrected} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \tag{3.3}$$

Tangential Distortion

$$x_{corrected} = x + [2p_1 xy + p_2(r^2 + 2x^2] \tag{3.4}$$

$$y_{corrected} = y + [p_1(r^2 + 2y^2) + 2p_2 xy] \tag{3.5}$$

where:

- k_1, k_2, k_3 are radial coefficients,
- p_1, p_2 are tangential coefficients and
- $r^2 = x^2 + y^2$

In OpenCV Bradski (2000), these coefficients were estimated once using the `calibrateCamera` function via standard checkerboard calibration Zhang (2000), and were stored for all subsequent image correction operations. The Levenberg–Marquardt algorithm Levenberg (1944) minimizes the reprojection error between detected and projected checkerboard corners to produce the optimal intrinsic matrix K and distortion vector.

3.2.3 | Perception and Localization Module

The localization module operated in two phases: a one-time **Initialization Phase** that detected the static arena boundaries and computed the homography matrix, and a continuous **Runtime Phase** that applied this mapping to warp each incoming frame for the control policy. These phases are formalized in Algorithm 1.

3.2.4 | ArUco Marker Detection

The localization pipeline began with the detection of ArUco markers (Garrido-Jurado et al., 2014), to define the Area of Interest (AOI). We utilized the `DICT_5X5_100` dictionary (5x5 bit grid, 100 unique IDs), which offers a balance between decoding speed and false-positive rejection.

Marker Configuration and Corner Assignment

To resolve the ambiguity of the arena's orientation and prevent the coordinate system from mirroring or rotating if the camera placement changes, specific unique IDs were assigned to the four physical corners of the operational area. This fixed assignment acted as a ground truth for the homography estimation. The configuration was explicitly defined as follows:

1. Top-Left (TL): Marker ID 24
2. Top-Right (TR): Marker ID 42
3. Bottom-Right (BR): Marker ID 66
4. Bottom-Left (BL): Marker ID 70

Detection was performed on the undistorted frame, not the raw frame — to ensure that the resulting homography matrix is free of lens distortion artifacts. The detection pipeline proceeded as follows:

1. **Preprocessing:** The RGB frame was converted to grayscale.
2. **Candidate Extraction:** Adaptive thresholding segmented black borders; contours were filtered by convexity and square geometry.
3. **Bit Decoding:** The inner region of each candidate was divided into a grid and its binary pattern was decoded to recover the marker ID.
4. **Corner Refinement:** Corner positions were refined to sub-pixel accuracy using `cornerSubPix`.

The output was a set of four ordered corner coordinates from the undistorted image. It should be noted that one ArUco marker was attached on top of the robot to provide the geometric representation of the robot.

3.2.5 | Planar Homography Estimation

The geometric relationship between the camera's image plane and the physical floor is modelled as a Planar Homography Hartley and Zisserman (2003), an invertible projective transformation mapping image pixel coordinates $p = [u, v, 1]^T$ to world coordinates $P_w = [x, y, 1]^T$:

$$s[u, v, 1]^T = H\,[u, v, 1]^T \tag{3.6}$$

where H is a 3×3 matrix with 8 degrees of freedom. Given the four sorted ArUco corner correspondences from Section 3.2.4, H was solved analytically via the Direct Linear Transformation (DLT) algorithm Abdel-Aziz and Karara (2015) using OpenCV's `getPerspectiveTransform` function.

Perspective Warping (Runtime Application). During runtime, the pre-computed H was applied to every incoming frame via `warpPerspective`:

$$\text{Img}_{world} = \texttt{warpPerspective}(\text{Img}_{raw},\ H) \tag{3.7}$$

This transformation produced a rectified Bird's-Eye View (BEV) image in which the four ArUco corner markers aligned precisely with the image boundaries. As a result, the CNN received a geometrically consistent representation of the environment that was largely invariant to minor variations in camera position, orientation, and perspective distortion. Figure 3.7 illustrates the resulting BEV perspective generated from the four corner ArUco markers.

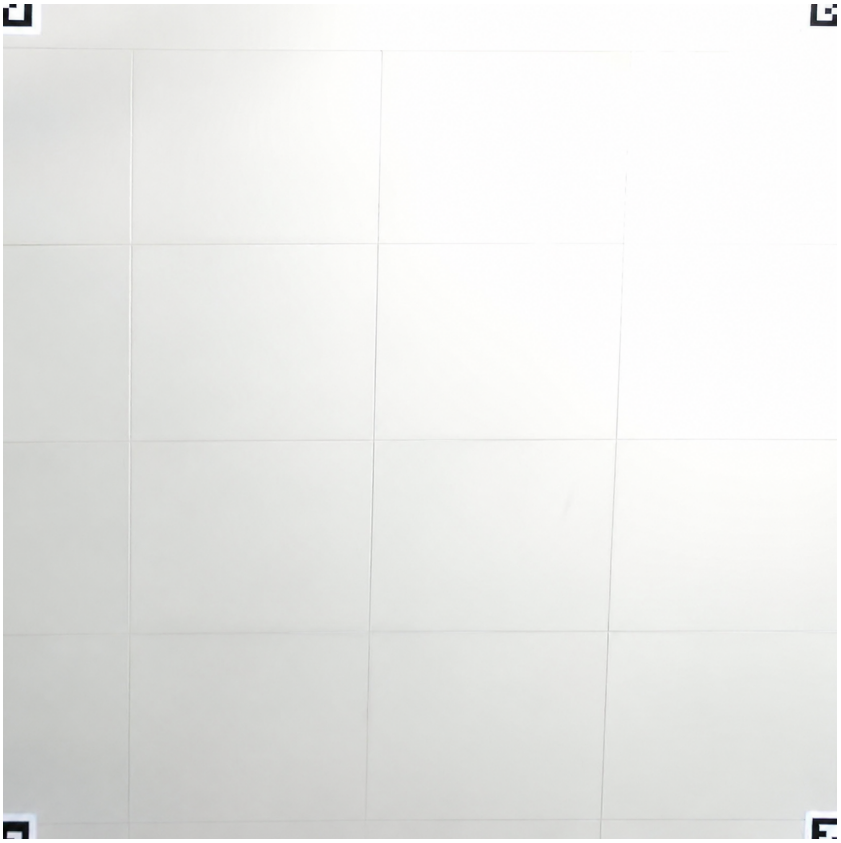


Figure 3.7: Rectified Bird's-Eye View (BEV) perspective generated using the four corner ArUco markers. The homography transformation maps the arena into a top-down view, providing a geometrically consistent input for the CNN.

3.3 | System Integration and Control Flow

The proposed system implemented a centralized, vision-based autonomous navigation architecture that offloaded all image processing and CNN inference to a remote workstation, communicating control commands to the robot over a local wireless link. The full architectural data flow is illustrated in Figure 3.8.

As depicted in Figure 3.8, the system architecture was divided into three functional subsystems: sensing, processing, and actuation. Together, these components formed a closed-loop control pipeline for autonomous navigation.

- **Sensing Subsystem (Arena):** Located in the physical environment, this block consisted of an overhead camera and fixed ArUco markers. The camera captured

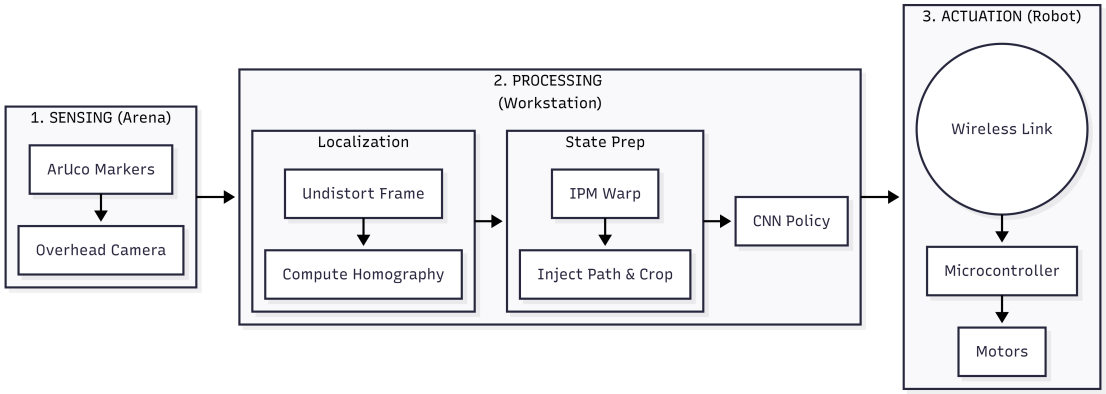


Figure 3.8: System Architecture.

the entire operational arena and provided the raw visual input used by the system. The ArUco markers acted as reference points for geometric correction and spatial alignment.

- **Processing Subsystem (Workstation):** This block contained the core intelligence of the system and executed a two-stage processing pipeline:
  - **Localization and State Preparation:** The system detected ArUco markers to compute a homography matrix, which warped the angled camera feed into a standardized top-down bird's-eye view using inverse perspective mapping (IPM). The transformed image was cropped and augmented with the target navigation path to form the system state.
  - **CNN-Based Policy Inference:** The pre-processed visual state was fed directly into a convolutional neural network (CNN), which served as an end-to-end behavioral policy mapping visual observations directly to low-level motor control commands.
- **Actuation Subsystem (Robot):** The inferred control commands, including steering and throttle signals, were transmitted wirelessly via Wi-Fi to the mobile robot. The robot's onboard microcontroller received these commands and drove the motors, completing the closed-loop control cycle.

3.3.1 | Module Integration

The software was organized into four distinct functional modules, ensuring separation of concerns and allowing independent testing of subsystems.

1. **Perception Interface (*CameraStream*):** Interfaced with the hardware camera driver and managed the frame buffer to ensure the processing pipeline always retrieved the most recent frame, minimizing input lag.

2. **State Estimator (*Preprocessor*):** Encapsulated the pre-computed homography matrix H from Section 3.2.5 and executed the full state representation pipeline — perspective warp, semantic extraction, path injection, egocentric crop, and normalization.

3. **Inference Engine (*PilotModel*):** Loaded the pre-trained CNN weights into memory on startup and exposed a `predict()` method that accepted a processed image tensor and returned the raw normalized steering command.

4. **Actuation Interface (*MotorDriver*):**

   - Abstracted low-level GPIO commands from the rest of the pipeline.
   - Converted the normalized CNN output into a raw steering command using a fixed gain and saturation, then mapped it to hardware-specific PWM duty cycles transmitted to the L298N motor driver.
   - Triggered a safety stop, defaulting all motors to zero velocity if the control loop was interrupted or the watchdog timer expired.

Algorithm 1 Homography Estimation and Perspective Warping

function INITIALIZEANDWARP($raw\_frame, camera\_matrix, dist\_coeffs, arena\_dims$)
  Input:
    raw_frame — Raw RGB image from the camera
    $camera\_matrix(K), dist\_coeffs(D)$ — Intrinsic calibration parameters
    $arena\_dims$ — Target width and height of bird's-eye view
  Output: $warped\_frame, homography\_matrix$

  // Phase 1: Undistortion
  $undistorted\_frame \leftarrow$ UNDISTORT(raw_frame, K, D)
  $gray\_frame \leftarrow$ CONVERTTOGRAY($undistorted\_frame$)

  // Phase 2: Marker Detection
  $dictionary \leftarrow$ GETDICTIONARY(`DICT_5X5_100`)
  $(corners, ids) \leftarrow$ DETECTMARKERS($gray\_frame, dictionary$)

  // Phase 3: ID-Based Sorting (Ground Truth Assignment)
  // Marker IDs: 0=TL, 1=TR, 2=BR, 3=BL
  $expected\_ids \leftarrow \{0, 1, 2, 3\}$
  if $|ids| < 4$ **or** $expected\_ids \not\subseteq ids$ **then**
    return Error "Markers 0–3 not found"
  end if
  $src\_points \leftarrow$ empty list of size 4
  for $i \leftarrow 1$ to $|ids|$ **do**
    if $ids[i] = 0$ **then**
      $src\_points[0] \leftarrow corners[i]$ ▷ Top-Left
    else if $ids[i] = 1$ **then**
      $src\_points[1] \leftarrow corners[i]$ ▷ Top-Right
    else if $ids[i] = 2$ **then**
      $src\_points[2] \leftarrow corners[i]$ ▷ Bottom-Right
    else if $ids[i] = 3$ **then**
      $src\_points[3] \leftarrow corners[i]$ ▷ Bottom-Left
    end if
  end for

  // Phase 4: Homography Calculation
  $dst\_points \leftarrow \{(0, 0), (W, 0), (W, H), (0, H)\}$
  $homography\_matrix \leftarrow$ GETPERSPECTIVETRANSFORM(src_points, dst_points)

  // Phase 5: Perspective Warping
  $warped\_frame \leftarrow$ WARPPERSPECTIVE($undistorted\_frame, homography\_matrix, arena\_dims$)
  return $warped\_frame, homography\_matrix$
end function

Algorithm 2 Navigation Agent: Neural Network Inference

function COMPUTECONTROLVECTOR($warped\_frame, cnn\_model$)
    Input: Top-down image and pre-trained CNN model
    Output: Steering angle and throttle commands

    // Step 1: Pre-processing
    $input \leftarrow$ RESIZE($warped\_frame, 200 \times 200$)
    $input \leftarrow$ NORMALIZEPIXELVALUES($input$) ▷ Scale to range [0, 1]

    // Step 2: Deep Learning Inference
    $(raw\_steer, raw\_throttle) \leftarrow$ MODELPREDICT($cnn\_model, input$)
    $k_{steer} \leftarrow 50$ ▷ Scaling gain from normalized model output to raw command
    $steering\_cmd \leftarrow k_{steer} \cdot raw\_steer$ ▷ Scale normalized steering
    $steer\_min, steer\_max \leftarrow -50, 50$ ▷ Raw command saturation bounds

    // Step 3: Safety Saturation
    ▷ Clip outputs to ensure values are within hardware limits
    $steering \leftarrow$ CLIP($steering\_cmd, steer\_min, steer\_max$)
    $throttle \leftarrow$ CLIP($raw\_throttle, 0.0, 1.0$)
    return $steering, throttle$
end function

Algorithm 3 Actuation: Hardware Control Interface

function DRIVEROBOT($steering, throttle$)
    Input: Normalized control commands from Neural Network
    Output: Physical electrical signals to motors

    // Step 1: Safety Check
    if Watchdog Timer Expired **then**
        TRIGGEREMERGENCYSTOP()
        return
    end if

    // Step 2: Signal Conversion
    ▷ Convert normalized values to Pulse Width Modulation (PWM)
    $servo\_signal \leftarrow$ MAPTOSERVODUTYCYCLE($steering$)
    $motor\_signal \leftarrow$ MAPTODCMOTORSPEED($throttle$)

    // Step 3: Hardware Execution
    WRITEGPIO(SERVO_PIN, $servo\_signal$)
    WRITEGPIO(MOTOR_PIN, $motor\_signal$)
end function

3.3.2 | Real-Time Execution and Middleware

The system operated using a synchronous main control loop. Unlike asynchronous systems that rely on complex middleware (e.g., ROS) for message passing, this study utilized a direct memory access approach to minimize overhead and latency. This design choice is critical for achieving stable end-to-end controller without PID dampening.

Execution Flow The main execution pipeline followed a "Sense-Think-Act" cycle:

1. **Initialization Phase:** On startup, the system verified hardware connections, warmed up the camera sensor (allowing auto-exposure to settle), and loaded the CNN model into RAM.
2. **The Control Loop:** The system entered a `while(true)` loop that executed the following sequence:
   - Acquire: The `CameraStream` fetched the latest frame.
   - Process: The `Preprocessor` warped and cropped the image.
   - Infer: The `PilotModel` computed the control vector.
   - Drive: The `MotorDriver` executed the command.
   - Log: (Optional) Data was saved for debugging or further training.

Timing and Latency To maintain smooth motion, the software was designed to run at a target frequency of 20 Hz. The most computationally expensive step was the CNN inference; therefore, the input image resolution was kept minimal to mitigate latency. During physical deployment the telemetry logger wrote one row per new camera frame at this 20 Hz recording rate, and Chapter 4 reports per-frame durations using this rate. Pixel-to-cm conversions throughout the thesis use the confirmed arena scale of 0.25 cm/px (800 px canvas $\approx$ 200 cm). A software watchdog was implemented to monitor the loop time; if the camera disconnected or the loop hung, the actuation interface triggered a safety stop to prevent runaway behavior.

3.4 | Data Acquisition and Processing

With the perception and localization pipeline established in the preceding sections, the system transitioned from passive environment sensing to active knowledge

acquisition. This phase operationalized the Behavioral Cloning paradigm described in Section 2.7.3, which constructed the supervised dataset that served as the sole source of driving knowledge for the Convolutional Neural Network. The data acquisition pipeline was designed to ensure that every recorded sample is a semantically meaningful, temporally synchronized pair of a preprocessed visual state and a human-demonstrated steering command. Telemetry frames were recorded frame-by-frame during deployment at an effective sampling frequency of approximately 20 Hz.

3.4.1 | Dataset Collection

A dataset of approximately 9,000 images was generated through a series of manual driving sessions in which a human operator controlled the differential-drive robot using a gamepad controller while the overhead camera system and data logger ran concurrently in real time. Figure 3.9 shows sample images collected during these manual driving sessions.

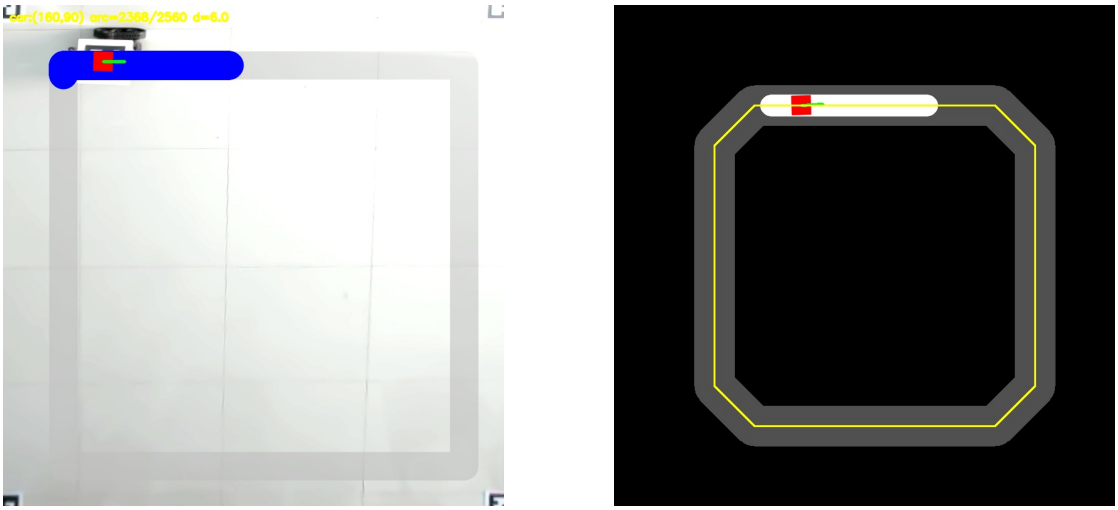


Figure 3.9: Sample images from the behavioral cloning dataset.

3.4.1.1 | Teleoperation Setup

During collection sessions, the robot was driven remotely via a gamepad interfaced with the workstation through Python's `pygame` library. The left analog stick's horizontal axis was mapped to a normalized steering command $\hat{u}_t \in [-1.0,\ 1.0]$ and then scaled by a fixed steering gain to the raw command a_t (we use $k_{steer} = 50$, so $a_t = k_{steer} \cdot \hat{u}_t$, yielded

raw command units in approximately $[-50, 50]$). Negative values corresponded to left turns, positive values to right turns, and $a_t \approx 0$ indicated straight-line motion. Throttle was held at a fixed, conservative value throughout data collection, consistent with the deployment strategy described in Section 3.5.4, so that the network was trained exclusively as a steering regressor.

3.4.1.2 | Synchronization Protocol

Temporal alignment between the visual state and the control action was the most critical requirement for producing a valid behavioral cloning dataset. A mismatch of even a single frame can corrupt the steering label associated with a given visual observation, introducing noise that degrades the learned policy. To eliminate this ambiguity, a dedicated data logger script executed within the same synchronous control loop described in Section 3.6.2. On every iteration of the loop, the following operations were performed atomically before any data was written to disk:

1. The latest undistorted frame was fetched from the `CameraStream` buffer.
2. The `Preprocessor` module applied the full state representation pipeline (detailed in Section 3.4.2) to produce the final input tensor s_t.
3. The current gamepad axis reading was polled via `pygame.event.pump()` to obtain the instantaneous steering value a_t.
4. The pair $(s_t,\ a_t)$ was appended to a running log as a NumPy array and the corresponding raw steering float was recorded to a paired `.catalog` file, keyed by a shared frame index.

This within-loop atomicity guaranteed that every saved image corresponded to the exact steering command the operator applied at that moment, with no buffering delay between visual capture and control sampling.

3.4.1.3 | Dataset Balancing

A naïve collection strategy produced a dataset heavily biased toward straight-line driving, since the majority of any track consisted of straight segments. A model trained on such an imbalanced distribution will learn to output near-zero steering commands by default, a degenerate policy that performs poorly on turns and recovery scenarios. To counteract this, the dataset was deliberately structured across three driving modes:

- **Nominal Laps:** The operator drove the robot smoothly along the digitally injected reference path, covering straight segments, left turns, right turns, and the transitions between them.
- **Aggressive Turn Demonstrations:** Additional dedicated runs were recorded in which the operator repeatedly navigated sharp 90-degree and U-turn topologies at the boundary of the robot's turning radius, increasing the representation of high-magnitude steering commands.
- **Recovery Maneuvers:** The robot was intentionally displaced laterally from the center of the reference path and then corrected back by the operator. These samples are critical for generalization: they teach the network the corrective steering behavior required to recover from covariate shift during autonomous deployment, a known failure mode of pure behavioral cloning Codevilla et al. (2019).

The dataset distribution reflects the collected demonstrations without post-hoc resampling. Training data diversity is achieved primarily through the three driving modes described above: Nominal Laps, Aggressive Turn Demonstrations, and Recovery Maneuvers.

3.4.2 | State Representation

Before any recorded frame was passed to the neural network, either during training or inference, it had to go through a deterministic, multi-stage preprocessing pipeline that transformed the raw overhead camera output into a compact, geometrically meaningful state representation. The goal of this pipeline is to enforce a strict information bottleneck: the network must receive only the visual features that are causally relevant to the steering decision, with all spurious environmental correlates which include floor texture, ambient shadow, arena boundary color, and absolute spatial position. In this pipeline, it is then systematically eliminated.

The complete pipeline proceeded through the following stages.

3.4.2.1 | Stage 1 — Homography Warp (Bird's-Eye View Rectification)

3.4.2.2 | Stage 1 — Homography Warp (Bird's-Eye View Rectification)

The raw, undistorted frame from the overhead camera was transformed using the pre-computed homography matrix H (computed during the initialization phase described in Section 3.4.2) via OpenCV's `warpPerspective` function:

$$\text{Img}_{\text{world}} = \text{warpPerspective}(\text{Img}_{\text{raw}}, H, \text{arena\_dims}) \tag{3.8}$$

This produced a geometrically rectified, orthographic top-down view of the arena in which pixel distances correspond to fixed metric distances on the floor, and the four ArUco corner markers aligned exactly with the image boundaries. The output was a standardized $W \times H$ pixel canvas representing the full area of interest. Figure 3.10 illustrates the resulting bird's-eye view after the homography transformation was applied.

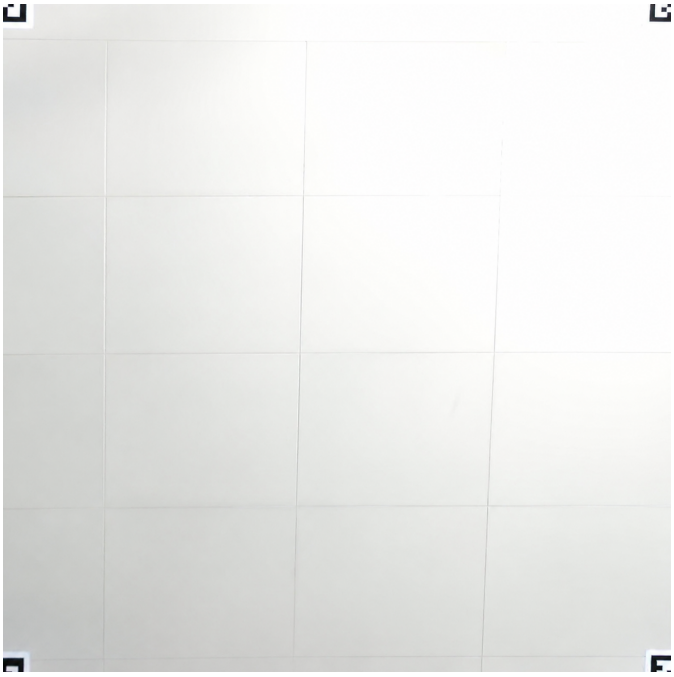


Figure 3.10: Rectified bird's-eye view of the arena after homography warping. The four ArUco corner markers are aligned to the image boundaries, establishing a geometrically consistent top-down representation of the navigation surface.

3.4.2.3 | Stage 2 — Semantic Polygon Extraction (Black Canvas Rendering)

3.4.2.4 | Stage 2 — Semantic Polygon Extraction (Black Canvas Rendering)

To decouple the network from the physical appearance of the arena floor, the drivable track boundaries within the warped image were isolated and re-rendered as discrete geometric polygons on a pure black canvas. The track boundaries were detected as contiguous regions of a known color or intensity within the warped frame and extracted using OpenCV's contour detection pipeline. The resulting polygons were drawn in white onto a zero-valued (black) background image of identical dimensions. Figure 3.11 shows the resulting semantic representation after polygon extraction was applied to the warped bird's-eye view.

This semantic abstraction ensured the following:

- All RGB texture, floor reflections, and lighting variation were discarded.
- The network received only the binary geometry of the drivable surface — the features that were invariant across sessions, lighting conditions, and track configurations.
- The model no longer processed an image of "a robot on a floor" but rather a normalized geometric representation of path curvature relative to the robot's position.

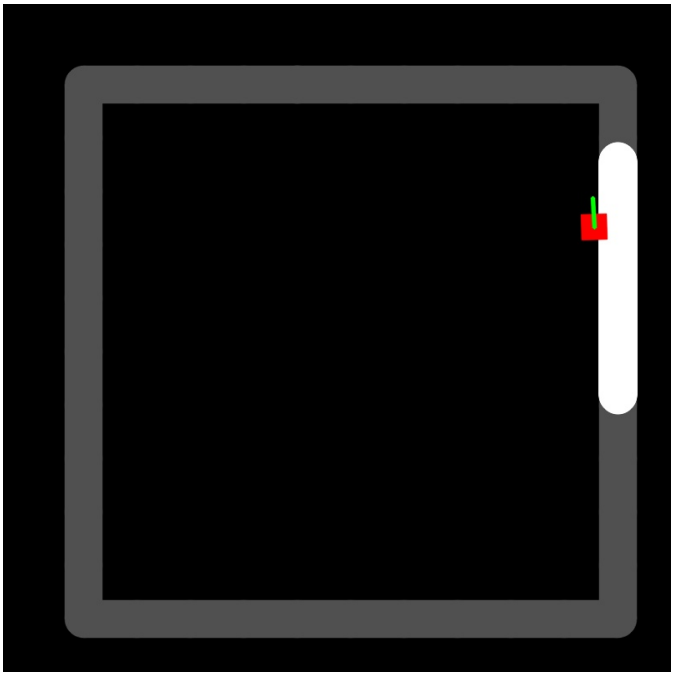


Figure 3.11: Semantic black canvas rendering of the drivable track boundaries. White polygons represent the extracted path geometry, with all floor texture, lighting artifacts, and background information discarded, leaving only the features causally relevant to the steering decision.

3.4.2.5 | Stage 3 — Path Injection (State Augmentation)

The semantic black canvas alone conveyed the shape of the drivable area but did not communicate the robot's navigation objective — the specific trajectory it was expected to follow within that area. To give the network directional intent, the target reference path was digitally rendered onto the black canvas as a second visual channel. The path was represented as a polyline or filled polygon, drawn in a distinct intensity value, that traced the desired centerline through the current track topology. The resulting composite image contained exactly two classes of information: the track boundaries (giving the network the constraint space) and the reference path (giving it the optimization target).

3.4.2.6 | Stage 4 — Egocentric Dynamic Cropping (ROI Extraction)

As established in the preliminary evaluation (Section 4.2), feeding the full warped arena into the CNN caused the network to overfit to the robot's absolute pixel coordinates rather than learning generalizable path geometry. The definitive solution was the

egocentric dynamic crop: at every time step, the robot's current pixel centroid (c_x, c_y) was detected from the chassis-mounted ArUco marker within the warped frame, and a fixed-dimension bounding window of size $W_{\text{crop}} \times H_{\text{crop}}$ (e.g., 200×200 pixels) was extracted centered on that coordinate:

$$s_t = \text{canvas}\left[c_y - \frac{H_{\text{crop}}}{2} : c_y + \frac{H_{\text{crop}}}{2},\ c_x - \frac{W_{\text{crop}}}{2} : c_x + \frac{W_{\text{crop}}}{2}\right] \tag{3.9}$$

By continuously re-centering the crop on the robot, its absolute position within the global arena was rendered invariant. From the network's perspective, the robot does not move; instead, the semantically extracted path geometry translated and rotated around the robot's fixed origin. The only dynamic variables passed to the convolutional feature extractor were the relative orientation, curvature, and proximity of the approaching path boundaries — the precise features required to generate a geometrically correct steering command. Figure 3.12 shows a representative sample of the egocentric crop as seen by the CNN during both training and inference.

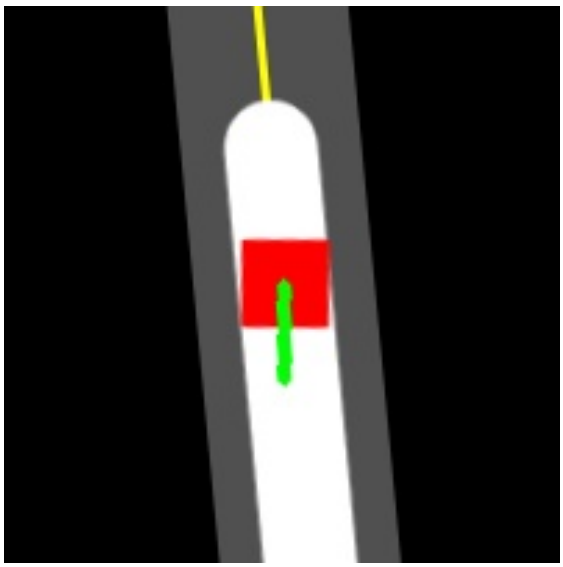


Figure 3.12: Representative egocentric dynamic crop fed to the CNN. The robot remains fixed at the center of the frame while the surrounding path geometry translates and rotates, ensuring the network learns steering behavior from relative path curvature rather than absolute spatial position.

3.4.2.7 | Stage 5 — Storage and Normalization

The cropped RGB canvas was stored as raw 8-bit integer values in the range $[0,\ 255]$; no division or floating-point conversion was applied prior to saving frames to disk. The CNN's dedicated input normalization layer applies the transformation at inference and training time, linearly scaling pixel values to the range $[-1.0,\ 1.0]$ via the expression:

$$x_{norm} = \frac{x}{127.5} - 1 \tag{3.10}$$

The output of Stage 5 is the final state tensor s_t — a $W_{crop} \times H_{crop} \times 3$ RGB array — that is saved to disk paired with its corresponding steering label a_t during data collection, and fed directly into the CNN's input layer during both training and deployment inference.

3.5 | Convolutional Neural Network Model and Training

This section details the implementation of the deep learning pipeline, translating the theoretical framework of Behavioral Cloning and CNNs (discussed in Section 2.7) into a functional navigation system. The pipeline is divided into model architecture design, data preprocessing, acquisition, training, and final deployment.

3.5.1 | Model Architecture

The study employed a modernized Convolutional Neural Network (CNN) architecture based on the NVIDIA DAVE-2 model Bojarski et al. (2016), optimized specifically for continuous regression tasks. Unlike classification networks that output probabilities for discrete categories, this network is designed to output continuous control values. The architecture follows a standard "encoder-decoder" style flow adapted for control, starting with an Input Layer that accepts full-colour RGB tensors of size $W_{crop} \times H_{crop} \times 3$ — the preprocessed egocentric crop described in Section 3.4.2.

The core processing begins with the Convolutional Feature Extractor, which utilizes five sequential convolutional layers to extract spatial features from the homography-transformed overhead view. The first three layers utilize 5x5 kernels with a stride of 2 to rapidly reduce spatial dimensions and capture broad geometric

structures, such as the path boundaries. The subsequent two layers employ 3x3 kernels with a stride of 1 to capture finer, higher-level spatial hierarchies. To mitigate the vanishing gradient problem and accelerate convergence, every convolutional layer is followed by Batch Normalization Ioffe and Szegedy (2015) and an Exponential Linear Unit (ELU) Clevert et al. (2016) activation function, which pushes mean activations closer to zero compared to standard ReLU.

Following feature extraction, a flattening operation converts the multidimensional feature maps into a 1D vector (containing 20,736 elements for a 200x200 input), preparing the data for high-level reasoning.

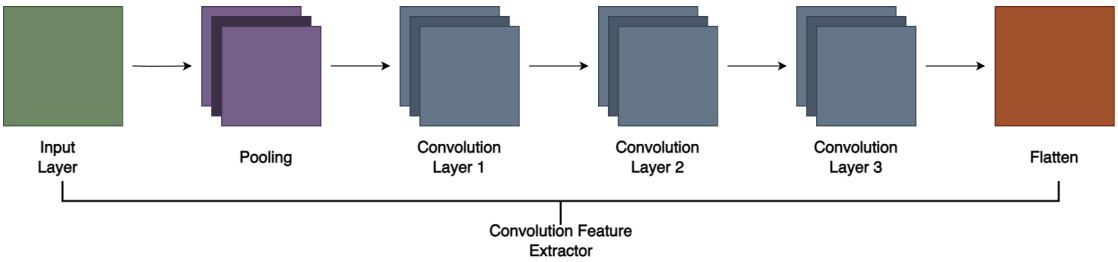


Figure 3.13: Feature Extraction Block: Input, Convolutional Layers, and Flattening

Once flattened, the data passes through the Fully Connected (Dense) Layers. Three hidden dense layers (containing 100, 50, and 10 neurons, respectively) interpret the extracted spatial features to serve as the decision-making bottleneck.These layers also utilize Batch Normalization and ELU activations. Furthermore, a mild Dropout rate of 10% (0.1) is applied to the first two dense layers to provide regularization and prevent overfitting, specifically tuned to avoid variance shifts when combined with Batch Normalization. The pipeline concludes with the Output Layer, consisting of a single neuron utilizing a Hyperbolic Tangent (Tanh) activation function. This function strictly binds the output to a [-1.0, 1.0] range, corresponding directly to a normalized turn-rate command that is converted to differential left and right wheel velocity targets by the *MotorDriver* module.

The total number of trainable parameters in the network is approximately 2,211,175, the majority of which reside in the first fully connected layer that maps the 20,736-element flattened vector to 100 neurons.

Layer	Type	Filters / Neurons	Kernel Size	Stride	Activation	Regularization
Input	Normalization	–	–	–	$(x/127.5) - 1$	–
1	Convolutional	24	5×5	2	ELU	Batch Norm
2	Convolutional	36	5×5	2	ELU	Batch Norm
3	Convolutional	48	5×5	2	ELU	Batch Norm
4	Convolutional	64	3×3	1	ELU	Batch Norm
5	Convolutional	64	3×3	1	ELU	Batch Norm
6	Flatten	20,736	–	–	–	–
7	Dense (FC)	100	–	–	ELU	Batch Norm, Dropout (0.1)
8	Dense (FC)	50	–	–	ELU	Batch Norm, Dropout (0.1)
9	Dense (FC)	10	–	–	ELU	Batch Norm
10	Dense (Output)	1	–	–	Tanh	–

Table 3.1: Table 3.1 Modified DAVE-2 Convolutional Neural Network Architecture

3.5.2 | Training Configuration

The network was trained as a single-output regression task, predicting only the normalized steering command $a_t \in [-1.0,\ 1.0]$. Throttle was excluded as a predicted output for reasons of training stability discussed at the end of this section.

Phase 1: Dataset Preparation

Prior to training, the collected dataset underwent a final preparation step. The full dataset was partitioned using an 85:15 split, with 85% of samples reserved for training and 15% held out for validation. To prevent the training loop from processing samples in a fixed sequence, which could introduce temporal autocorrelation artifacts, the training split was shuffled before each epoch. A batch size of 64 was used throughout training.

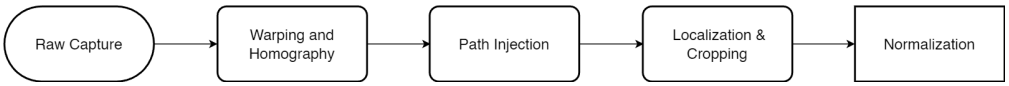


Figure 3.14: The image preprocessing pipeline converting raw camera input into a state representation for the CNN

Phase 2: Network Optimization

The optimization objective was to minimize the Mean Squared Error (MSE) between the predicted steering command and the human demonstrator's recorded command across all samples in each batch:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \left(y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)} \right)^2 \tag{3.11}$$

The Adam optimizer Kingma and Ba (2015) was selected for its adaptive learning rate capabilities, ensuring stable convergence despite the non-convex nature of the loss landscape. A ReduceLROnPlateau scheduler is applied to the Adam optimizer; when the validation loss stagnates across successive epochs, the learning rate is reduced to assist convergence. Training proceeds for the full scheduled number of epochs without early termination.

The throttle command was held at a fixed, conservative value during both data collection and inference rather than being predicted as a second CNN output. This design decision ensured training stability: in behavioral cloning, predicting two continuous outputs required the MSE loss to simultaneously balance two potentially conflicting gradient signals during backpropagation. Restricting the network to steering-only prediction provided a cleaner, more interpretable loss signal and allows the model to converge more efficiently. This approach also followed the precedent established by NVIDIA's PilotNet architecture, which originally focused exclusively on steering angle prediction with throttle managed as an independent system. It should be noted that while the data collection script records both steering and throttle labels as a paired array for logging completeness, only the steering column is used during training; the throttle label is discarded prior to model optimization.

3.5.3 | Deployment and Inference

The trained model was exported and optimized for the target hardware. The deployment process operated in a continuous control loop.

1. **Model Serialization:** The trained network architecture and weights are saved in PyTorch's native serialization format (`.pth`).
2. **Inference Loop:** The deployment script ran in a continuous loop on the dedicated offboard workstation, transmitting the resulting control commands to the robot's onboard microcontroller over a local Wi-Fi connection via UDP sockets:
    a. Captured and warped the overhead camera image using the homography transformation.

b. Preprocessed the image and extracted the Region of Interest (ROI).

c. Fed the processed sensor input into the Convolutional Neural Network (CNN).

d. The CNN output raw regression values corresponding to control commands (e.g., steering = 0.3, throttle = 0.8).

e. The predicted values were formatted into a lightweight data payload and transmitted over Wi-Fi via UDP sockets to the robot's Raspberry Pi. The onboard microcontroller then unpacked this data, scaled it to the appropriate PWM range (0–255), and transmitted the electrical signals directly to the motor drivers via GPIO

3.6 | Evaluation and Testing

To rigorously assess the performance of the end-to-end CNN controller, a series of controlled experiments were designed. These tests aimed to quantify the system's ability to track a digitally rendered path using only visual inputs, specifically evaluating whether the network has learned generalizable kinematic behaviors or merely memorized spatial coordinates. This section outlines the testing parameters, the track topologies, and the mathematical metrics used to validate the system.

3.6.1 | Physical Testing Environment and Setup

Because the preprocessing pipeline systematically eliminated ambient RGB textures, shadows, and background noise, the physical testing did not require a visually isolated robotics arena. Testing was conducted on a standard, flat indoor surface with consistent artificial lighting. The system was deployed following the environment setup as defined in Section 3.1.2.

3.6.2 | Evaluation Scenarios

The system was tested across three track configurations of increasing Bézier curve complexity, each designed to isolate specific kinematic behaviors and evaluate the CNN policy's adaptability. All tests were conducted in a controlled indoor environment with constant artificial lighting to minimize perception noise.

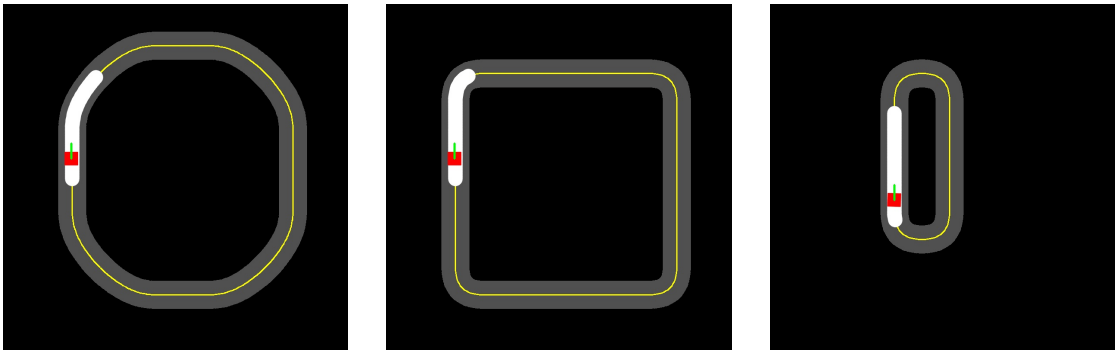
1. **Scenario 1 (45-Degree Bézier Curves):** This topology featured mild, continuous curvature to assess the model's baseline tracking stability. The primary goal of this test was to evaluate whether the CNN can generate smooth, continuous steering adjustments under gradual geometric changes without exhibiting oscillatory steering behavior (fishtailing) or overreacting to shallow curves.

2. **Scenario 2 (90-Degree Bézier Corners/Square Path):** This scenario tested the model's response to rapid, orthogonal geometric transitions. Navigating a square path with rounded corners evaluated the network's ability to map sudden geometric shifts to aggressive, high-magnitude steering commands and rapidly recover heading and straight-line stability upon corner exit.

3. **Scenario 3 (180-Degree Sustained U-Turns):** This loop featured extreme, continuous curvature to test the system's kinematic endurance under prolonged geometric distortion. The scenario evaluated the network's ability to scale and sustain near-maximum steering angles for extended durations, and smoothly transition through extreme curve sections.

4. **Scenario 4 (Held-Out Track):** A fourth track topology (`path-new`; Figure 3.16) that was *not* present in the training set was evaluated to test generalization to unseen path geometry within the same marker arena. This scenario isolated whether the policy learned transferable relative path geometry rather than memorizing track-specific coordinates.

Each scenario is evaluated over $N = 10$ independent trials. Every trial begins from a randomized start pose within the lane and constitutes a single discrete circuit, in contrast to the earlier single continuous multi-lap recordings. This randomized-start, discrete-trial protocol yields independent samples of tracking performance per scenario, enabling trial-to-trial repeatability analysis and confidence-interval reporting in Chapter 4.



(a) Scenario 1: 45-Degree Curves.

(b) Scenario 2: 90-Degree Corners.

(c) Scenario 3: 180-Degree U-Turns.

Figure 3.15: Visual representation of the three trained experimental track configurations used for physical testing.

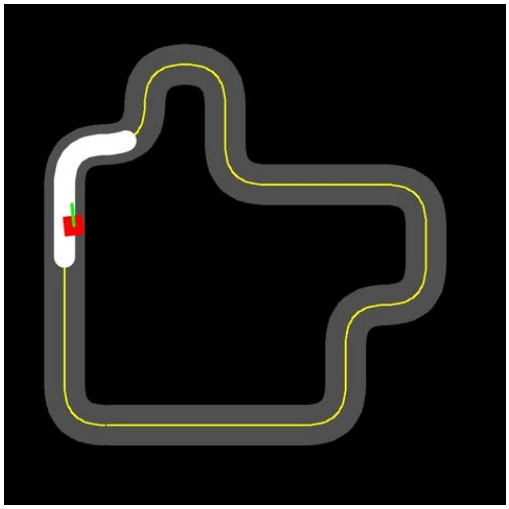


Figure 3.16: The held-out track (`path-new`) used for the Scenario 4 generalization test. This topology was *not* present in the training set and is evaluated under the same randomized-start protocol as the trained tracks.

3.6.3 | Ground-Truth Tracking System

To accurately measure the real-world kinematic performance of the robot without relying on manual estimation, the system utilized an automated error-logging script during physical deployment. By tracking the robot's chassis-mounted ArUco marker relative to the digitally injected reference path, the script directly computed instantaneous error metrics frame-by-frame at an effective sampling frequency of

approximately 20 Hz during deployment.

Unlike traditional SLAM-based evaluations that require logging global (X, Y) spatial coordinates and absolute yaw angles, this lightweight evaluation pipeline recorded only the direct relative deviations between the robot and the reference path. This approach minimized data overhead while providing a precise, continuous measurement of the controller's tracking accuracy throughout each trial.

3.6.4 | Performance Metrics

The evaluation metrics were divided into two distinct categories: theoretical metrics utilized during the offline supervised learning phase, and real-world kinematic metrics utilized during physical deployment.

3.6.4.1 | Theoretical Metrics

These metrics were calculated exclusively during the offline training phase of the CNN. They measured the mathematical variance between the human-provided steering angles from the behavioral cloning dataset and the model's predicted steering angles.

The **Mean Squared Error (MSE)** served as the primary loss function during backpropagation. It heavily penalized large deviations between predicted and actual steering values:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^{N} \left(y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)} \right)^2 \tag{3.12}$$

The **Mean Absolute Error (MAE)** provided a linear, easily interpretable measure of the average steering angle error in normalized units during the validation phase:

$$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^{N} \left| y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)} \right| \tag{3.13}$$

3.6.4.2 | Real-World Kinematic Metrics

Because theoretical metrics can be deceptive due to factors such as covariate shift or spurious background correlations, the ultimate validation of the system relied on physical deployment metrics. These measured the actual spatial performance of the

differential-drive robot on the floor across all four evaluation scenarios defined in Section 3.6.2.

Cross-Track Error (CTE) and RMSE. The fundamental measure of tracking accuracy is the Cross-Track Error (CTE), defined as the signed perpendicular distance between the robot's geometric center and the nearest point on the digital reference path at any given time step t. Formally, given the robot's center (c_x, c_y), the nearest point on the path (p_x, p_y), and the unit path tangent vector (t_x, t_y) at that point, the signed CTE is computed as:

$$CTE_t = \frac{t_x(c_y - p_y) - t_y(c_x - p_x)}{\sqrt{t_x^2 + t_y^2}} \tag{3.14}$$

where:

- (c_x, c_y) is the robot's geometric center in warped-frame pixel coordinates,
- (p_x, p_y) is the nearest point on the digital reference path,
- (t_x, t_y) is the unit tangent vector of the path at that nearest point.

A positive value indicates the robot is to the right of the path direction; a negative value indicates it is to the left. To provide a single summary statistic for an entire run, the Root Mean Square Error (RMSE) of the per-frame CTE deviations is calculated as:

$$RMSE = \sqrt{\frac{1}{N}\sum_{t=1}^{N} CTE_t^2} \tag{3.15}$$

A lower RMSE indicates tighter adherence to the reference path. CTE was measured in warped-frame pixel units, which were geometrically consistent across all arena configurations since the homography transformation always mapped the arena to a fixed output resolution regardless of physical area dimensions.

Heading Error. Complementing the position error was the Heading Error, which evaluated the robot's orientation accuracy. This metric measured the angular difference between the robot's current yaw angle θ_{robot}, derived from the ArUco marker's top-edge midpoint vector, and the tangent angle of the path θ_{path} at the nearest projected point:

$$\psi_t = \theta_{robot} - \theta_{path} \tag{3.16}$$

The result was wrapped to the range $[-\pi, \pi]$ to ensure sign consistency, and was reported in degrees for interpretability. Monitoring heading error is critical for predictive analysis, as high angular deviations often precede a complete departure from the lane even when the instantaneous CTE remains low.

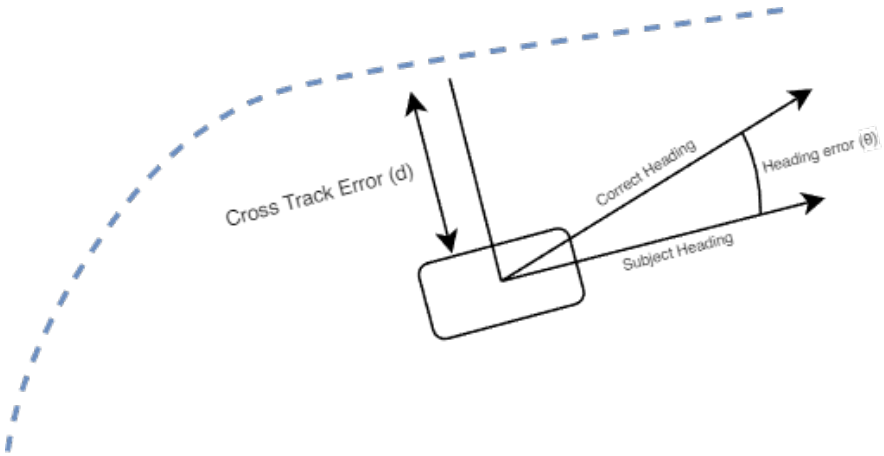


Figure 3.17: Visual representation of Cross-Track Error (d) and Heading Error (θ).

Success Rate. Finally, the overall reliability of the system was measured by the **Success Rate**, defined as the percentage of total attempts in which the robot completed the entire course without any part of its chassis crossing the lane boundaries:

$$\text{Success Rate} = \frac{\text{Number of Successful Laps}}{\text{Total Attempts}} \times 100\% \tag{3.17}$$

A trial was considered failed when the chassis-mounted ArUco marker was no longer detected within the drivable region for more than 30 consecutive frames, corresponding to approximately 1.5 seconds at the system's measured operating rate of 20 Hz.

The success rate is reported per scenario over the $N = 10$ trials. For any trial that does not complete, the **completion fraction** is also reported, defined as the maximum path-segment index reached divided by the number of path points minus one.

4

Results & Discussion

This chapter presents the evaluation of the developed end-to-end autonomous navigation system. The results are presented chronologically, tracing the system's iterative development. All physical deployment and inference runs reported in this chapter were executed on the real robot, with the trained model running onboard a Raspberry Pi embedded on the platform.

To validate the necessity of the final image preprocessing architecture, the chapter first details the offline training and real-world physical testing of two early experimental baselines. By analyzing the specific failure modes of these early iterations, the empirical justification for the proposed Egocentric Dynamic Crop Model is established.

4.1 | Chronological Development and Experimental Baselines

During the development of the navigation framework, the state representation of the track underwent three major iterations. All models were trained using a modified NVIDIA DAVE-2 Convolutional Neural Network (CNN) architecture with 2,211,175 parameters, processing 200×200 pixel images, and minimizing Mean Squared Error (MSE) using the Adam optimizer.

4.1.1 | Iteration 1: Baseline A (Allocentric Full AOI Model)

The initial approach utilized the raw, top-down Bird's-Eye View (BEV) image, retaining the room's floor textures and permanent background elements.

During offline training, Baseline A converged rapidly. As shown in Figure 4.1, both MSE and MAE decreased exponentially, with validation loss plateauing near 0.05 and 0.13, and reaching a low validation MSE of 0.0489 by Epoch 30.

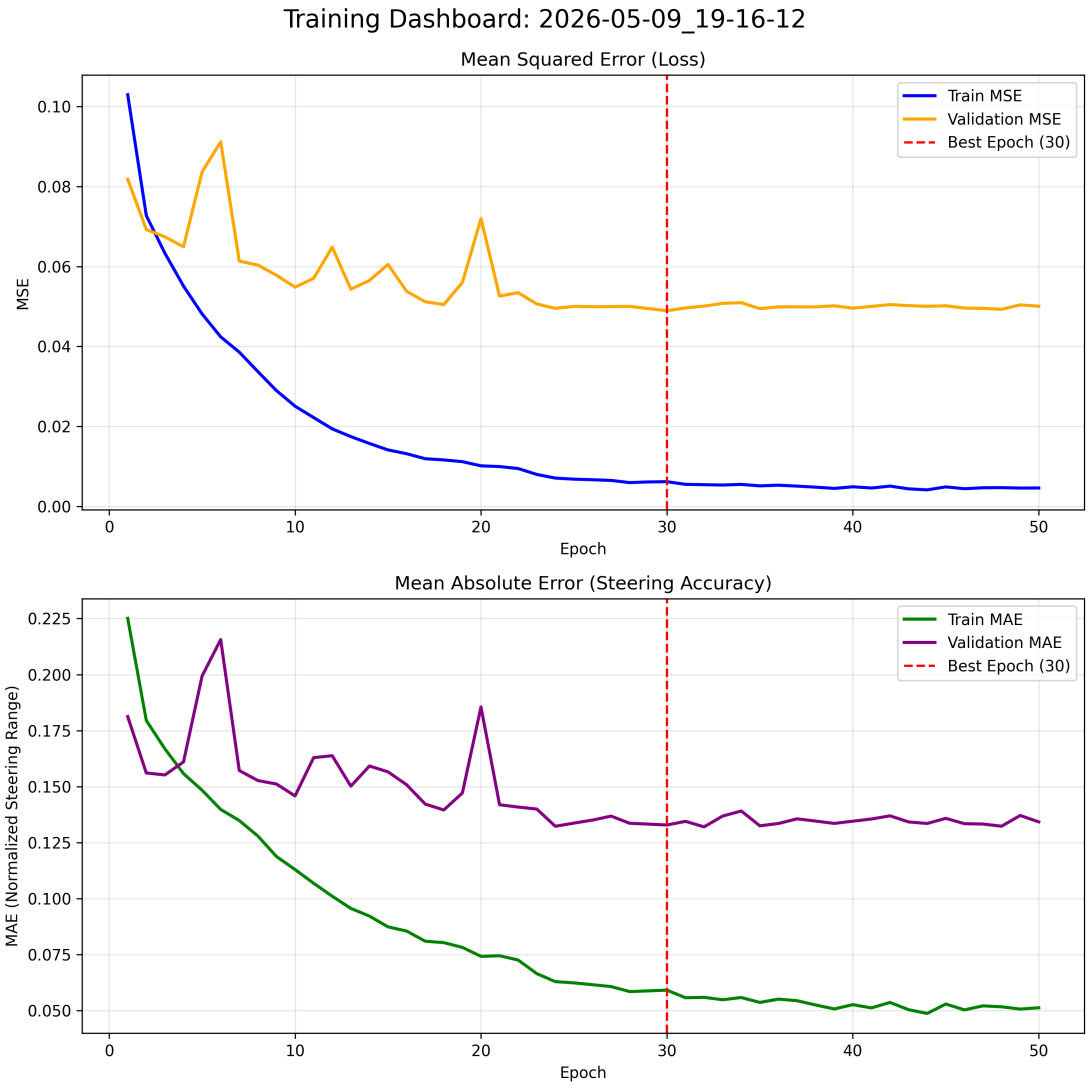


Figure 4.1: The near-perfect convergence of MSE and MAE suggested successful learning, though this was later identified as spatial memorization.

However, physical deployment resulted in immediate, catastrophic navigation failure. On the physical track, the robot exhibited erratic motor control, immediately veering off-path and breaching the arena boundaries.

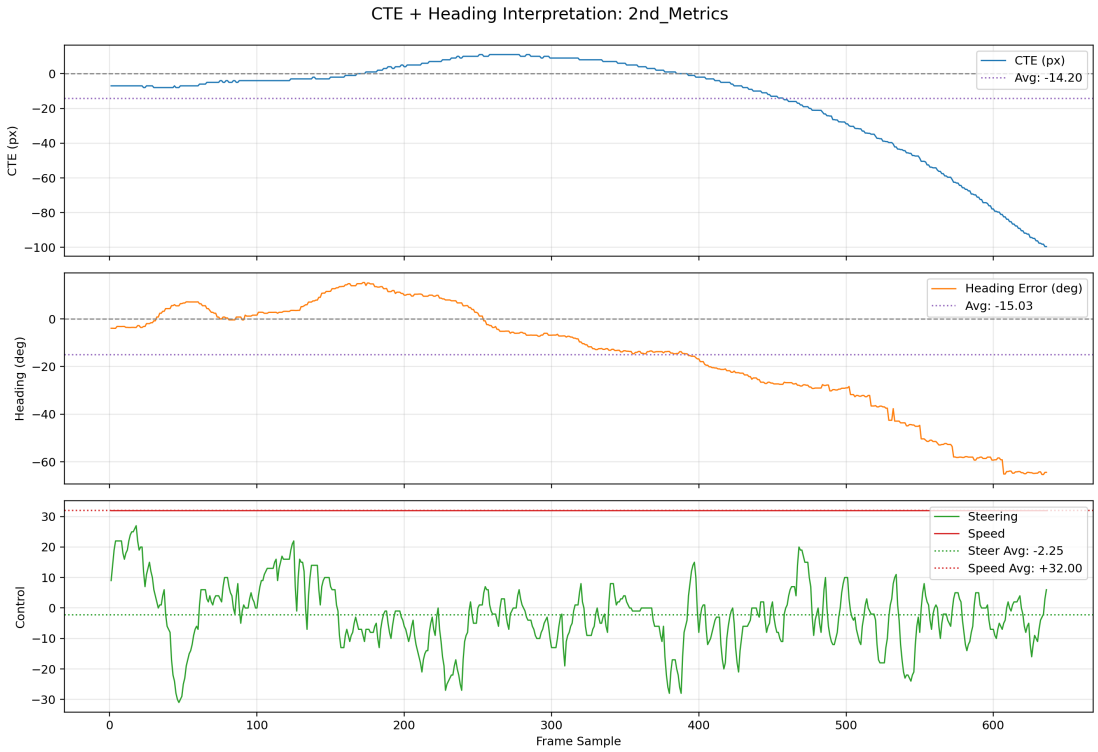


Figure 4.2: Physical deployment of the Allocentric Full AOI Model resulted in immediate navigation failure.

As shown in Figure 4.2, both the Cross-Track Error (CTE) and the Heading Error (θ_{err}) diverged immediately on deployment (CTE quickly exceeding -25 cm and Heading Error dropping past -60°). Baseline A maintained the path for only 636 frames with an RMSE of 7.75 cm. When analyzing the Error Compounding Rate over the first 500 frames, Baseline A showed rapid, exponential error growth. Minor starting position changes on the real track confused the network, causing it to immediately steer off course.

Post-deployment analysis revealed the model suffered from **shortcut learning**. Because the room environment and global track remained static in the camera's view, the CNN bypassed learning the path geometry entirely. Instead, it overfit directly to the robot's absolute pixel coordinates within the room frame. The validation error

remained low during training because validation images came from that same static frame. During inference, any minor starting deviation created an out-of-distribution pixel state that the model could not interpret.

4.1.2 | Iteration 2: Baseline B (Feature-Extracted Allocentric Model)

To eliminate background shortcuts like floor reflections, shadows, and pixel discrepancies, the pipeline was updated to add a semantic polygon extraction step. This process isolated the AOI as flat white shapes with a red polygon on a pure black canvas, removing all environmental noise and forcing the network to learn the path geometry itself.

While this removed environmental noise, offline training metrics immediately destabilized. As shown in Figure 4.3, the training profile exhibited severe validation noise and a massive loss spike near epoch 13, indicating the network struggled to map steering angles without its familiar background shortcuts. The resulting controller was not stable enough to justify full physical deployment, and the model was therefore not advanced beyond limited sanity checks.

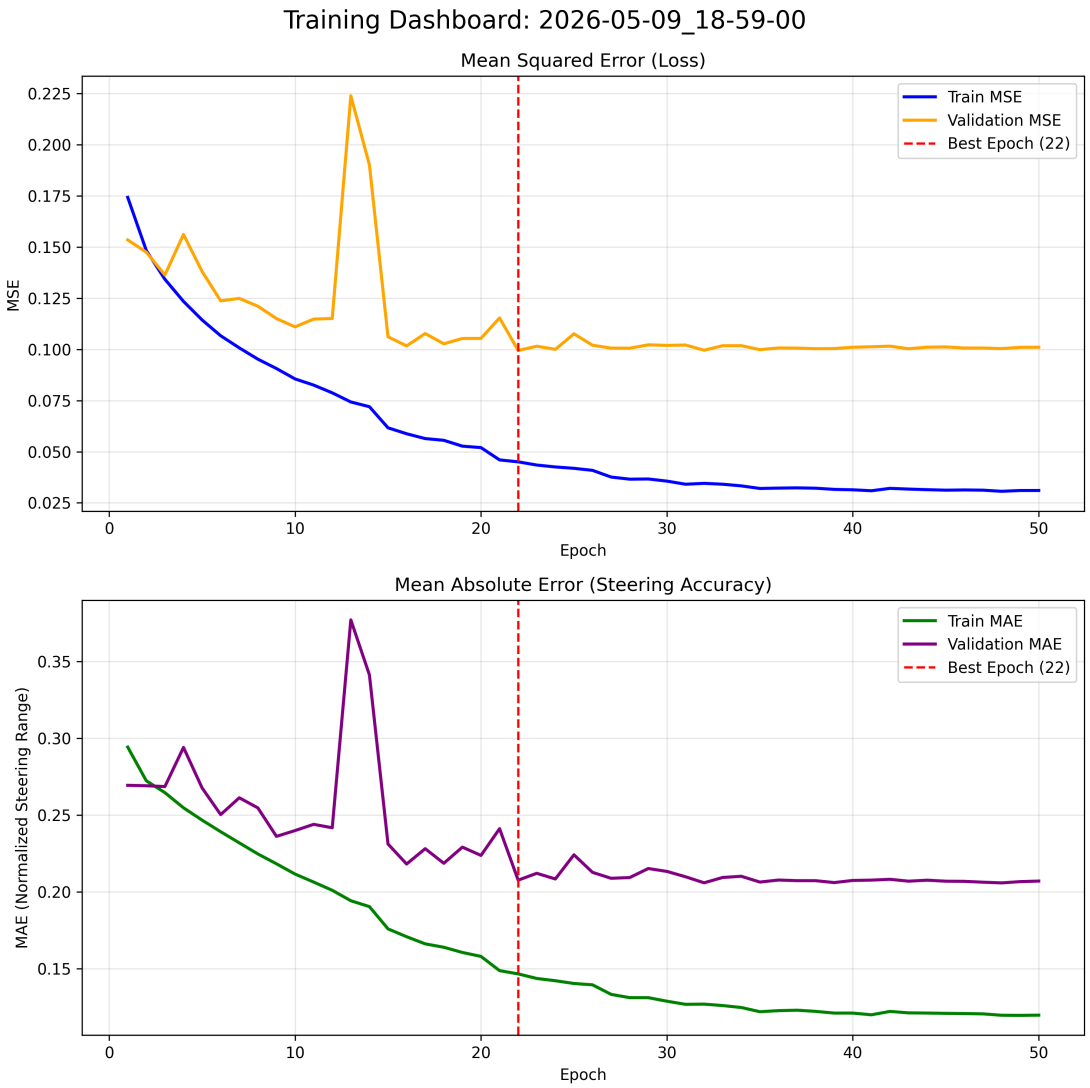


Figure 4.3: The Feature-Extracted Allocentric Model's training profile was unstable, with severe validation noise and a massive loss spike, indicating the network struggled without background shortcuts.

Limited real-world trials at reduced speed still exposed the same instability. The failure was reproduced across two separate deployment sessions, in which the controller survived only 772 and 1,122 frames with CTE RMSE values of 8.93 cm and 21.09 cm respectively (Table 4.1), confirming that the instability was systematic rather than an isolated event. As shown in Figure 4.4 for the longer session, the Cross-Track Error (CTE) drifted steadily upward to roughly 20 cm before collapsing, while the

Heading Error hovered around -20° and then jumped abruptly to approximately $+60^\circ$ near the failure point. The steering signal oscillated with a near-zero mean while speed was increased, indicating the controller was reacting to noise rather than tracking a stable path.

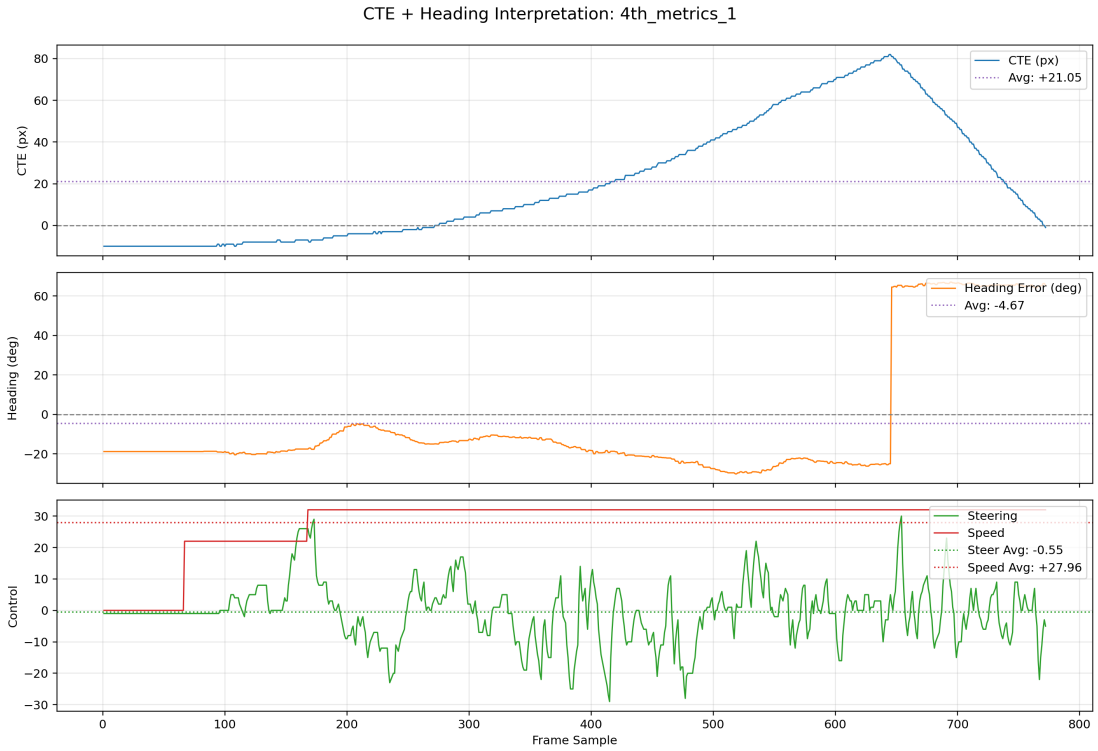


Figure 4.4: Limited real-world trial for Baseline B. CTE drifted upward with a late-stage heading discontinuity, while steering oscillations persisted under increasing speed.

Failure Analysis of Baselines A and B

The two allocentric baselines failed for complementary reasons that both trace back to the state representation. Baseline A preserved the full room context, which enabled shortcut learning: the CNN exploited static background cues and absolute pixel coordinates rather than learning the path geometry. This produced excellent offline metrics but catastrophic real-world generalization under even minor pose shifts.

Baseline B removed the environmental shortcuts, but the allocentric frame still did not encode the robot's egocentric heading and local path curvature in a stable way. The mapping from a global, top-down AOI to a precise steering command became

ambiguous and noise-sensitive, leading to unstable training dynamics and a controller that could not be stabilized for deployment.

In both deployment sessions the robot immediately departed the path boundary and exhibited no corrective steering response, confirming that the instability observed during training reflected a fundamental inadequacy of the allocentric representation rather than a tuning problem.

Together, these failures established the need for a representation that is both egocentric and dynamically aligned to the robot's local path context, motivating the subsequent Egocentric Dynamic Crop Model.

4.2 | The Egocentric Dynamic Crop Model

To resolve both the shortcut learning of Iteration 1 and the spatial translation issues of Iteration 2, the final proposed model implemented the Egocentric Dynamic Crop.

By cropping the image continuously around the ArUco marker, the robot's position remains fixed at the center of the neural network's view. With the background removed, the path geometry simply rotates and approaches the robot, forcing the CNN to learn actual driving rules based strictly on path curvature.

4.2.1 | Offline Training Convergence

The proposed model resolved the previous instability issues. By centering the image on the robot, the network required only 9,679 samples to establish geometric stability.

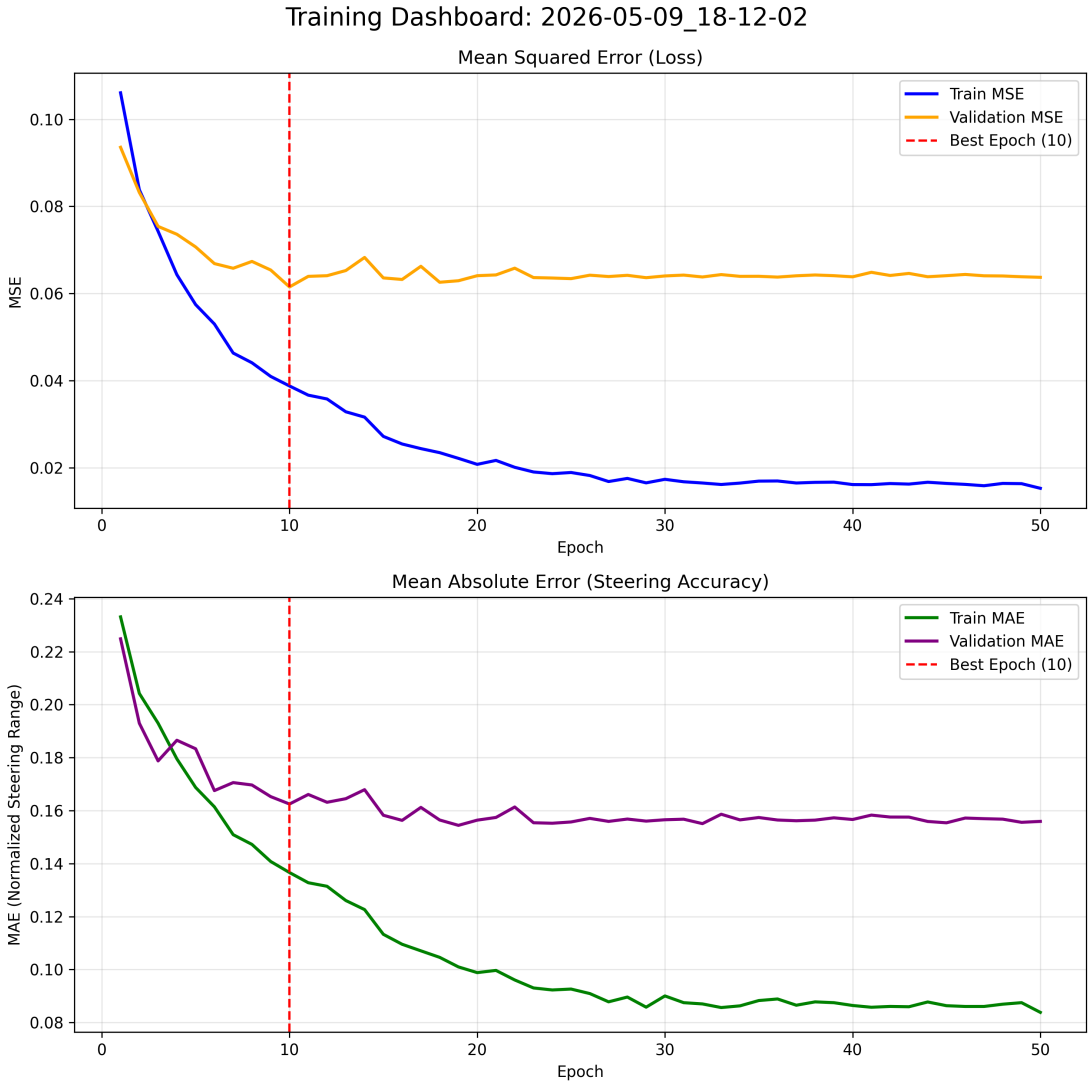


Figure 4.5: The Egocentric Dynamic Crop Model's training profile showed stable convergence, with both MSE and MAE decreasing steadily without significant validation noise.

As shown in Figure 4.5, it quickly achieved a highly stable validation MSE of 0.0615 by Epoch 10. The entire optimization process was completed in just 2 minutes and 24 seconds, proving the model is both accurate and computationally lightweight.

4.2.2 | Real-World Navigation and Endurance

The proposed Egocentric Model successfully guided the robot through the entire track.

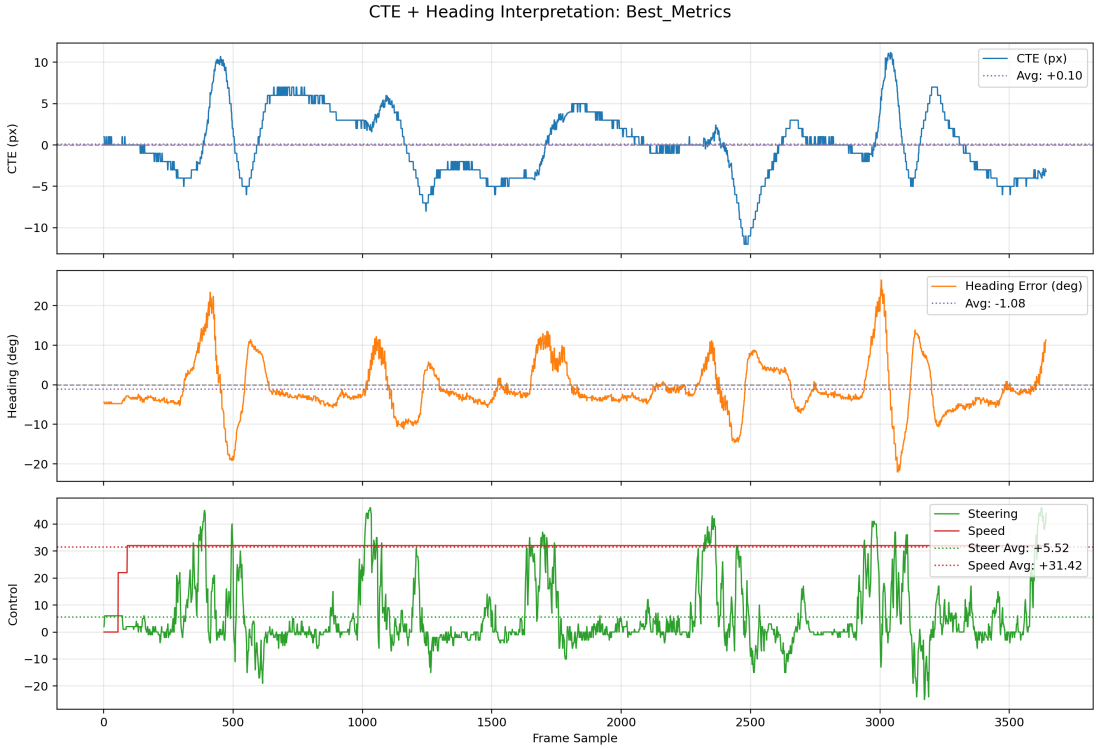


Figure 4.6: Physical deployment of the Egocentric Dynamic Crop Model resulted in successful navigation through the entire track, with stable CTE and Heading Error profiles.

Deployed over a continuous run of 3,642 frames, it achieved an extremely low tracking error with an RMSE of 1.01 cm and a Mean Absolute CTE of just 0.79 cm. The telemetry in Figure 4.6 demonstrates smooth, consistent steering across both straightaways and sharp turns without a single boundary breach.

4.3 | Comparative Tracking Performance

Table 4.1 and Figure 4.7 summarize the final real-world tracking metrics across all three chronological iterations, clearly illustrating the dramatic stability improvements achieved by the egocentric crop.

Table 4.1: Physical Track Testing Results Across All Iterations

Physical Trial	Active Model	Total Frames Survived	Track Outcome	CTE (RMSE, cm)
Baseline A	Baseline A (Full AOI)	636	Drove off track	7.75
Baseline B (Session 1)	Baseline B (Feature-Extracted)	772	Drove off track	8.93
Baseline B (Session 2)	Baseline B (Feature-Extracted)	1,122	Drove off track	21.09
Final System	Proposed Egocentric Model	3,642	Completed successfully	1.01

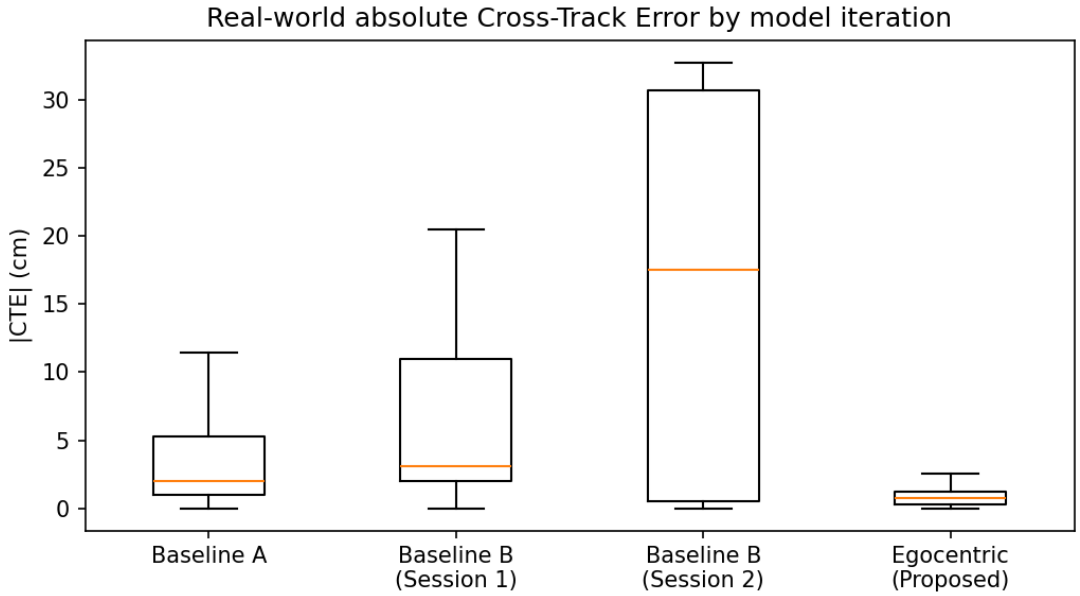


Figure 4.7: Real-World Trajectory Deviations: absolute Cross-Track Error (cm) box plot across model iterations.

The box plot in Figure 4.7 shows a dramatic contraction of the error distribution for the proposed model, with a much lower median and tighter interquartile range

than Baselines A and B. This indicates not only lower average tracking error but also substantially reduced variability and fewer large deviations.

4.4 | In-Depth Performance of the Model

To comprehensively validate the proposed end-to-end architecture, the Egocentric Dynamic Crop model was evaluated across three trained track topologies of increasing Bézier curve complexity, plus a fourth held-out topology reserved for the generalization test (Section 4.4.7). By entirely removing the PID control components and relying strictly on the Convolutional Neural Network (CNN) for high-level navigation, this phase of testing isolates the controller's ability to dynamically scale continuous steering outputs based solely on visual geometric features.

4.4.1 | Statistical Analysis Methodology

For each evaluation scenario, the robot was deployed in $N = 10$ independent trials. Each trial begins from a randomized start pose within the lane and constitutes a single discrete circuit of the track, in contrast to the earlier single continuous multi-lap recordings. During every trial the tracking system recorded per-frame Cross-Track Error (CTE) and Heading Error at an effective control-loop rate of approximately 30 Hz (mean 29.7 Hz, SD 1.6 Hz across the 40 trials). Because each trial is exactly one circuit, tracking consistency is reported across the independent trials (trial-to-trial repeatability) rather than over laps within a single run. Success rates are reported as the proportion of completed trials per scenario. The full quantitative summary is presented in Tables 4.2, 4.4, and 4.3.

All cross-track errors are measured in the rectified bird's-eye-view canvas, which spans 800×800 pixels and maps to the physical arena of approximately 200 cm (2.0 m) per side, yielding a confirmed scale of 0.25 cm per pixel. This factor is used consistently throughout this chapter and Chapter 3 to convert pixel measurements to approximate real-world distances. The steering command reported throughout is the raw controller command after scaling the network's normalized output (scaling gain $k_{steer} = 50$, clipped to $[-50, 50]$), where positive values denote rightward turns and negative values leftward turns. Commands between -2 and 2 are treated as straight-line driving (deadband). Empirically, this raw command maps monotonically to the robot's measured yaw rate and saturates at high-magnitude commands as the

differential-drive platform reaches its motor limits; the values are therefore reported as unitless command magnitudes rather than physical angles.

Table 4.2: Kinematic Performance Summary of the Final Egocentric Model, pooled over $N = 10$ independent randomized-start trials per scenario.

				Cross-Track Error (cm)				Heading (°)	
Sc.	**Track Topology**	N	**Frames/trial**	**Mean ± SD**	**RMSE**	**P95**	**Max**	**Mean ± SD**	**P95**
1	45° Bézier curves	10	1,266	0.79 ± 0.57	0.97	1.88	3.05	1.25 ± 6.73	13.55
2	90° square path	10	1,514	1.28 ± 0.78	1.50	2.63	4.25	1.38 ± 7.54	18.73
3	180° sustained U-turn	10	950	0.76 ± 0.51	0.92	1.68	2.52	2.21 ± 6.70	14.43
4	Held-out track (unseen)	10	1,674	1.64 ± 1.23	2.05	4.00	6.24	1.29 ± 10.96	23.45

Frames/trial is the mean number of telemetry frames per trial (one circuit). All cross-track errors are reported in centimetres using the confirmed arena scale 0.25 cm/px (1 px = 0.25 cm).

4.4.2 | Trial-to-Trial Tracking Consistency

Because each evaluation trial is exactly one circuit of the track, the repeated unit is the *independent trial* rather than a lap within a continuous run. Tracking consistency is therefore reported as the distribution of per-trial CTE RMSE across the $N = 10$ randomized-start trials of each scenario (Table 4.3).

Table 4.3: Trial-to-trial CTE RMSE consistency ($N = 10$ independent randomized-start trials per scenario).

Scenario	**Trials (n)**	**Per-trial RMSE mean ± SD (cm)**	**Range (cm)**
1 (45°)	10	0.98 ± 0.11	0.76–1.15
2 (90°)	10	1.49 ± 0.14	1.26–1.69
3 (180°)	10	0.91 ± 0.04	0.86–0.96
4 (held-out)	10	2.04 ± 0.16	1.77–2.44

The small standard deviations (0.04–0.17 cm) are the central result: the controller produces near-identical tracking error across independent trials that each start from a different randomized pose, indicating high repeatability rather than a single lucky run. The trained-track scenarios (1–3) cluster tightly, while the held-out track (Scenario 4) shows both a higher mean and a wider spread, consistent with the generalization gap reported in Section 4.4.7.

4.4.3 | System Reliability

Across all four scenarios the egocentric controller completed every trial without the chassis leaving the lane: no failures were observed in $4 \times 10 = 40$ independent randomized-start trials (Table 4.4). Given the modest sample of $N = 10$ trials per scenario, we report this conservatively as "no failures observed across 10 independent randomized-start trials per scenario" rather than claiming near-perfect reliability in an absolute sense.

Table 4.4: Closed-loop reliability per scenario ($N = 10$ randomized-start trials each).

Scenario	N	Successes	Success rate	Mean completion
1 (45°)	10	10	100.0%	100.0%
2 (90°)	10	10	100.0%	100.0%
3 (180°)	10	10	100.0%	100.0%
4 (held-out)	10	10	100.0%	100.0%

4.4.4 | Scenario 1: 45-Degree Bézier Curves

The first scenario tests mild, continuous curvature across $N = 10$ independent randomized-start trials (mean 1,266 frames per trial). Rather than exhibiting the fishtailing or oscillatory steering behavior commonly observed in pure behavioral cloning models without derivative damping, the model demonstrated highly anticipatory steering. As the 45° curves entered the egocentric bounding box, the convolutional feature extractor registered the gradual geometric shift, resulting in a smooth, continuous adjustment of the steering vector. Pooled over the 10 trials, the model achieved a mean absolute CTE of **0.79** $\pm$ **0.57** cm (RMSE = 0.97 cm), with 95% of all frames remaining within 1.88 cm of the reference path. Trial-to-trial consistency was high: per-trial CTE RMSE was **0.98** $\pm$ **0.11** cm (range 0.76–1.15 cm, Table 4.3), confirming near-identical tracking across independent runs. The Heading Error remained bounded, with a mean of $1.25 \pm 6.73°$ and a 95th-percentile magnitude of **13.55**°, showing that the model does not overreact to gentle path deviations and smoothly tracks shallow curves.

4.4.5 | Scenario 2: 90-Degree Square Path

Navigating the square path across $N = 10$ independent trials (mean 1,514 frames per trial) tested the model's response to rapid, orthogonal geometric changes. Pooled over the 10 trials, the model achieved a mean absolute CTE of **1.28** $\pm$ **0.78** cm (RMSE =

1.50 cm), with 95% of frames within 2.63 cm of the reference path and a worst-case deviation of 4.25 cm. The Heading Error remained bounded, with a mean of $1.38 \pm 7.54°$ and a 95th-percentile magnitude of 18.73°. Per-trial CTE RMSE was **1.49** $\pm$ **0.14** cm (range 1.26–1.69 cm, Table 4.3). Approaching a 90° corner requires a sharp spike in actuation, and upon entering the apex the model executed aggressive, high-magnitude steering commands, reaching raw command magnitudes near the ± 50 saturation limit (peak ≈ 46, with 95% of commands at or below a magnitude of 32).

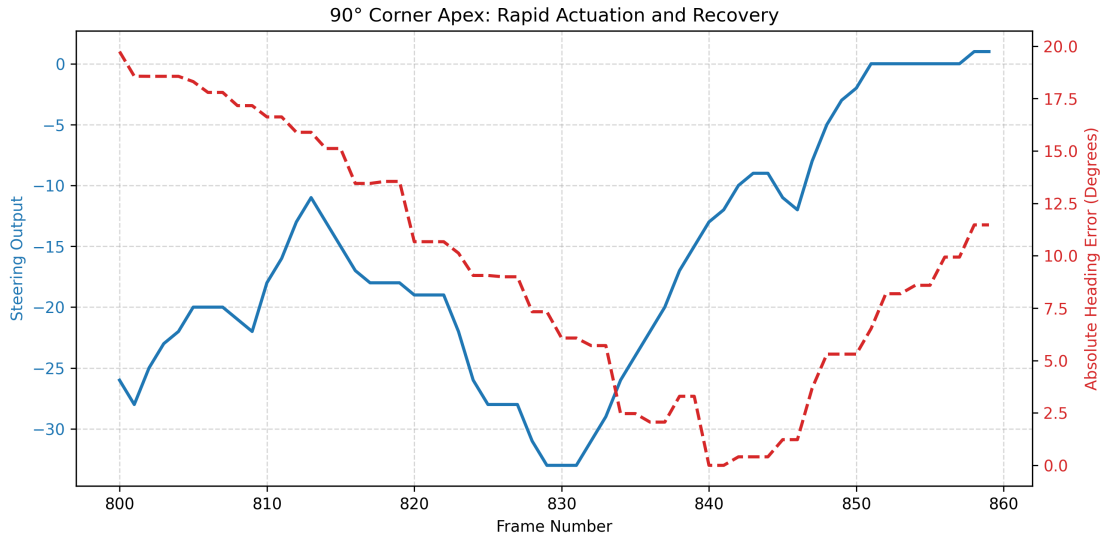


Figure 4.8: 90-degree cornering telemetry showing rapid actuation and recovery under orthogonal curvature.

Importantly, as visualized in Figure 4.8, the Egocentric Dynamic Crop ensured that the path boundaries remained centered in the network's view rather than sweeping out of frame. This continuous recentering allowed for a rapid and stable recovery: upon exiting each corner the model returned the heading error below 5° within roughly one second (median 1.2 s across the 90° corner exits, measured at the $\approx$ 30 Hz loop rate), resuming straight-line stability.

4.4.6 | Scenario 3: 180-Degree Sustained U-Turns

The final trained topology tested the system's kinematic behavior under extreme, continuous curvature. The scenario was evaluated across $N = 10$ independent trials (mean 950 frames per trial), each a single circuit of the U-turn track. During the

180-degree U-turns, the model was forced to hold a near-maximum steering angle through the sustained curve.

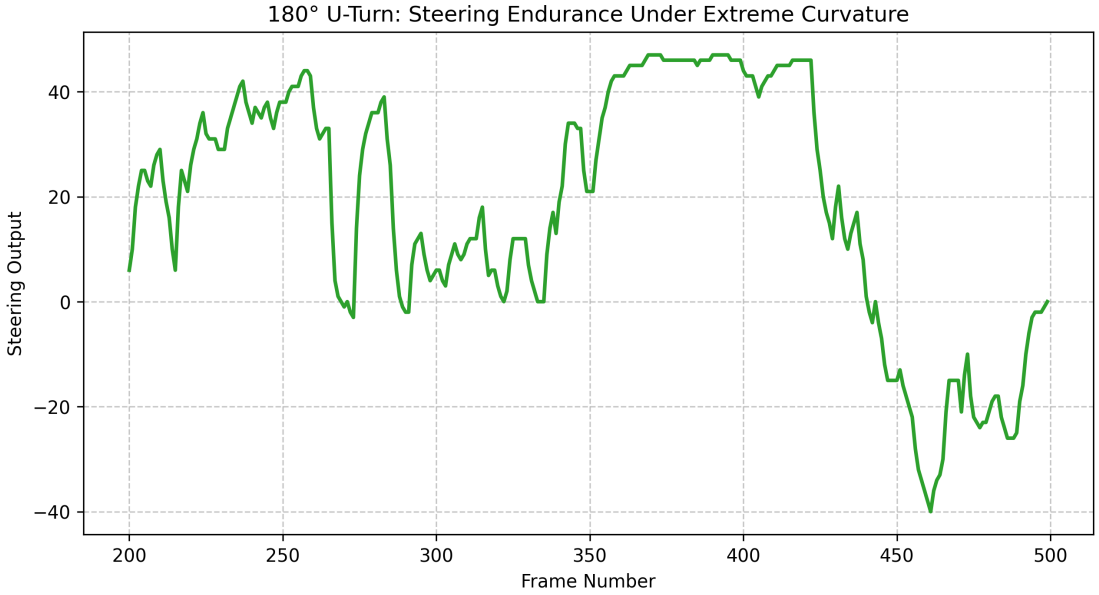


Figure 4.9: 180-degree U-turn telemetry illustrating sustained steering actuation under extreme curvature.

The telemetry in Figure 4.9 confirms that the network successfully handled the extreme geometric distortion, scaling up its actuation to sustain steering through the curve (95% of commands at or below a magnitude of 34, peaking near the ± 50 saturation limit). Pooled over the 10 trials, the model achieved a mean absolute CTE of **0.76** $\pm$ **0.51** cm (RMSE = 0.92 cm), with 95% of frames within 1.68 cm of the reference path and a worst-case deviation of 2.52 cm, well below half the chassis width. The Heading Error had a mean of $2.21 \pm 6.70^{\circ}$ with a 95th-percentile magnitude of 14.43°. Trial-to-trial consistency was the tightest of all scenarios: per-trial CTE RMSE was **0.91** $\pm$ **0.04** cm (range 0.86–0.96 cm, Table 4.3), demonstrating that tracking performance remained bounded and highly repeatable despite the extreme curvature. This confirms that the CNN effectively learned the structural rules of extreme curvature tracking without requiring a rigid mathematical path-planning algorithm.

4.4.7 | Generalization to an Unseen Track Topology

The decisive test of whether the network learned transferable path geometry rather than memorizing track-specific coordinates is its behavior on a track it never saw during training. The held-out track (`path-new`) was evaluated under the identical $N = 10$ randomized-start protocol.

Pooled over the three trained tracks (Scenarios 1–3, $N = 30$ trials), the controller achieved a CTE RMSE of **1.20 cm** and a mean absolute CTE of 0.98 cm, with all 30/30 trials successful. On the held-out track ($N = 10$), CTE RMSE rose to **2.05 cm** with a mean absolute CTE of 1.64 cm, while success was maintained at 10/10. Tracking error on the unseen topology is therefore 1.70× **the trained-track RMSE, with no loss of closed-loop reliability** (Figure 4.10).

This pattern — a moderate, bounded increase in tracking error but zero failures on an unseen track — is consistent with the model having learned *relative path geometry that transfers within the structured marker arena*, rather than a track-specific coordinate map. We scope this claim to unseen track *topologies within the marker arena*; it does not extend to unstructured or markerless environments, which Chapter 1 explicitly places outside the scope of this work.

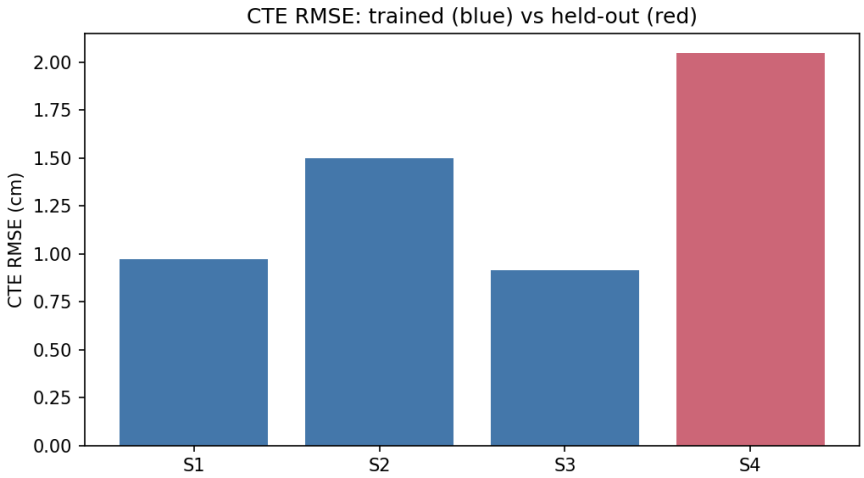


Figure 4.10: Generalization: CTE RMSE on the trained tracks (pooled, $N = 30$) versus the held-out track ($N = 10$). Error grows 1.70× on the unseen topology while success remains at 100%.

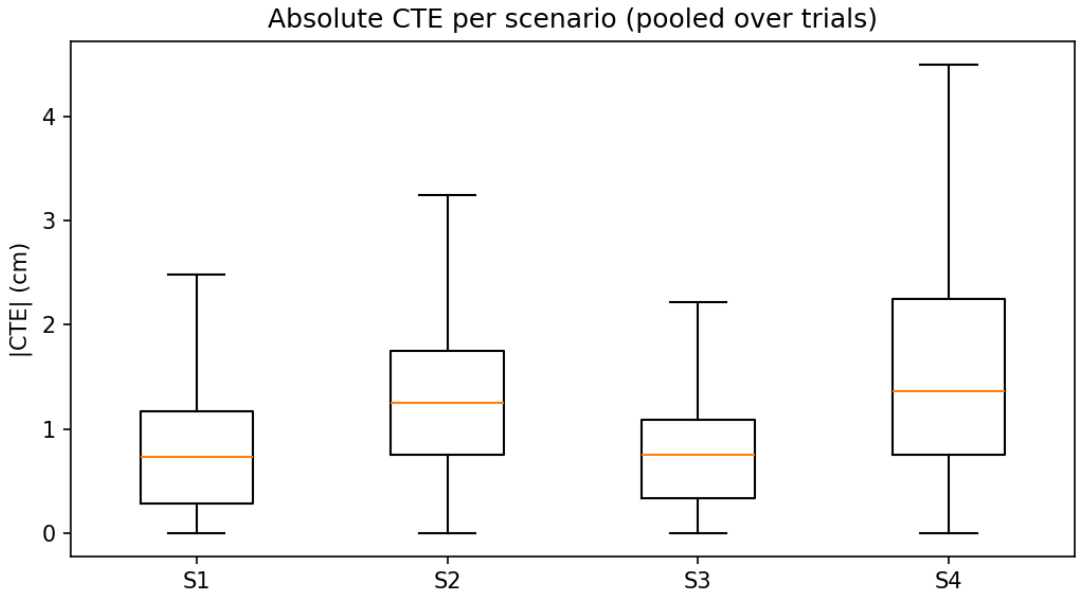


Figure 4.11: Distribution of absolute Cross-Track Error (cm) per scenario, pooled over $N = 10$ trials each, including the held-out track. Error scales with curvature severity and rises on the unseen topology while remaining bounded.

4.5 | Control Decision Discretization and Confusion Matrix

To examine the directional structure of the steering decisions issued by the end-to-end CNN, the continuous telemetry from all 40 evaluation trials was discretized into three control states and cross-tabulated. Under the system's sign convention, positive values denote a rightward sense for both quantities: a positive CTE indicates the chassis is displaced to the *right* of the reference path, and a positive steering command drives a rightward turn. Consequently, a positive CTE calls for a corrective *leftward* steering action. The desired (corrective) action is defined from the lateral displacement (CTE) using a 0.5 cm (2 px) dead-band:

$$\text{Desired Action} = \begin{cases} \text{Left}, & \text{if CTE} > 0.5 \text{ cm} \\ \text{Straight}, & \text{if } -0.5 \text{ cm} \leq \text{CTE} \leq 0.5 \text{ cm} \\ \text{Right}, & \text{if CTE} < -0.5 \text{ cm} \end{cases} \tag{4.1}$$

The executed action is discretized from the raw steering command with a matching ± 2 command-unit dead-band: commands above $+2$ are classified as Right, those below -2 as Left, and the interval between as Straight. Cross-tabulating the desired action (rows) against the executed action (columns) over all 54,031 marker-tracked frames yields the contingency table in Table 4.5, visualized as a row-normalized heat map in Figure 4.12; in both, the diagonal corresponds to instantaneous agreement between the lateral-error correction and the executed steering.

Table 4.5: Control-decision contingency table pooled over all four scenarios (40 trials, 54,031 marker-tracked frames). Rows are the desired corrective action implied by the instantaneous CTE; columns are the executed steering action. Cells give frame counts with row percentages in parentheses; bold marks the diagonal (instantaneous agreement).

	Executed steering action			
Desired (from CTE)	**Left**	**Straight**	**Right**	**Total**
Left	**2,228 (22.2%)**	3,072 (30.6%)	4,726 (47.1%)	10,026
Straight	2,630 (16.9%)	**6,374 (40.9%)**	6,580 (42.2%)	15,584
Right	5,765 (20.3%)	9,508 (33.5%)	**13,148 (46.3%)**	28,421
Total	10,623	18,954	24,454	54,031

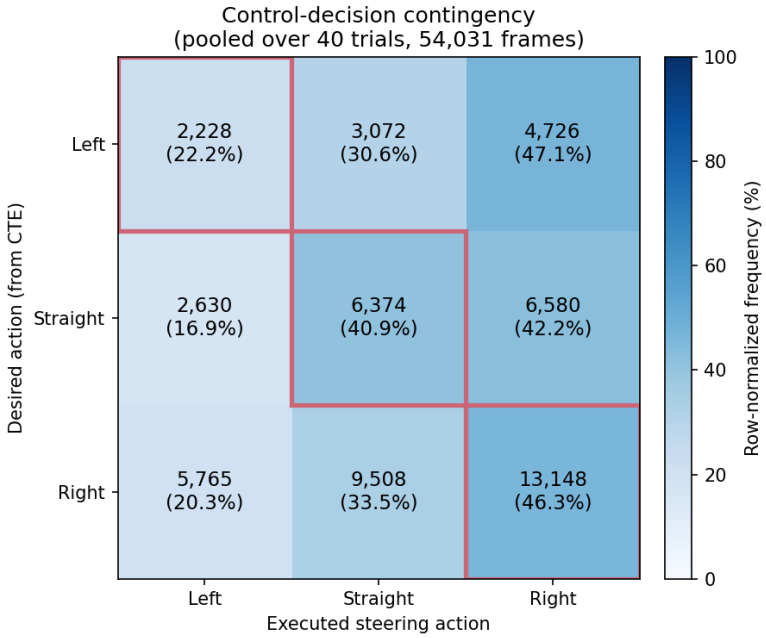


Figure 4.12: Control-decision contingency heat map (pooled over all four scenarios, 54,031 marker-tracked frames). Rows are the desired corrective action implied by the instantaneous CTE and columns are the executed steering action; cells are shaded by row-normalized frequency and annotated with frame counts and row percentages. The diagonal cells (instantaneous agreement) are outlined. The darker Right column reflects a mild rightward steering bias, while the substantial off-diagonal mass reflects anticipatory, phase-lagged steering rather than tracking failure.

If the controller implemented a purely reactive proportional correction, the table would concentrate on its diagonal. Instead, only 40.3% of frames lie on the diagonal, and even when both the desired and executed actions are committed turns (excluding the dead-band) the directional agreement is 59.4%. This modest instantaneous agreement is expected, and is consistent with the rest of the chapter for two reasons.

First, **closed-loop phase lag**. A frame-level regression of the raw steering command on the instantaneous heading error and on the CTE yields a coefficient of determination below 0.1 in both cases: the steering signal leads while the lateral error responds roughly half a second later. A strict frame-by-frame comparison therefore registers as disagreement even though the loop is stable and convergent over time.

Second, **anticipatory curve following**. Much like a human driver, the network begins to turn while still on the straight section just before a curve, rather than waiting

until a lateral error has accumulated. During this lead-in the chassis is still close to the centerline (so the instantaneous CTE implies "Straight" or even the opposite correction) while the executed steering is already committed to the turn. The resulting off-diagonal mass therefore reflects this look-ahead behaviour rather than tracking error. This behavior is consistent with findings from human driving research. Lehtonen et al. observed that drivers make look-ahead fixations toward upcoming sections of a curve before substantial steering corrections are required, using visual information from the future road segment for trajectory planning rather than relying solely on immediate lane-position errors. Experienced drivers exhibited these anticipatory behaviors more frequently, indicating that successful curve negotiation depends on previewing upcoming path geometry rather than purely reactive controlLehtonen et al. (2014).

Crucially, this low instantaneous agreement coexists with the 100% closed-loop success rate and sub-centimetre tracking reported above. The contingency table should therefore be read not as a classifier accuracy but as direct evidence that the CNN learned a temporally extended, anticipatory steering policy rather than an instantaneous proportional-correction law—the same conclusion reached from the steering–error regressions. The table also reveals a mild rightward steering bias: the Right column holds 45.3% of all executed actions, and the single largest cell is the correct Right–Right pairing (13,148 frames), corresponding to the controller steering right to recover from a slight leftward offset that the chassis carried through most trials.

5

Conclusions and Recommendations

5.1 | Summary of Findings

This study developed and evaluated a lightweight, end-to-end autonomous navigation framework for a differential-drive robot operating in a structured indoor environment. The system replaced traditional sensor-heavy approaches such as LiDAR-based SLAM and multi-sensor fusion with a single overhead RGB camera and four ArUco fiducial markers. A planar homography matrix was dynamically computed at runtime to transform the angled camera feed into a geometrically rectified Bird's-Eye View (BEV) representation of the arena. A modernized DAVE-2 Convolutional Neural Network (CNN), trained exclusively through Behavioral Cloning on human-demonstrated steering data, then mapped this preprocessed visual state directly to continuous steering commands without any intermediate PID feedback or classical planning algorithm.

Physical deployment initially revealed a critical failure mode: the allocentric (full-AOI) state representation caused the network to rely on the robot's absolute pixel position rather than learn generalizable path-following features. This behavior is consistent with shortcut learning, where a model exploits an easy but unintended predictive feature that performs well under training conditions yet fails to generalize under more challenging or real-world conditions. This issue was mitigated through an architectural pivot to an egocentric perception model, wherein the CNN received only a robot-centered crop drawn on a semantically abstracted black canvas. This information bottleneck forced the convolutional layers to learn translation-invariant relative geometric features.

The resulting egocentric model successfully navigated the 45-degree Bézier, 90-degree square, and 180-degree U-turn topologies, as well as a held-out track never seen during training, demonstrating that homography-based preprocessing combined with a strict visual information bottleneck can substantially bridge the gap between offline training loss and real-world kinematic performance.

5.2 | Conclusion

This study conclusively demonstrates that a lightweight end-to-end autonomous navigation framework using homography-based overhead localization over an egocentric semantic state representation approach can achieve reliable autonomous navigation on a resource-constrained robotic platform. The research directly addressed the central problem statement: the absence of an accessible, low-cost autonomous navigation framework that operates exclusively on camera input in a structured planar environment.

The proposed system fulfilled this requirement without resorting to expensive range sensors, probabilistic mapping pipelines, or high-end onboard computing hardware. A central achievement of this work was the successful elimination of classical PID feedback loops from the control architecture. The trained DAVE-2 CNN produced sufficiently smooth and accurate continuous steering outputs to maintain stable path following across all tested track topologies, including geometrically demanding 90-degree turns and hairpin U-turns.

This validates the core hypothesis of end-to-end Behavioral Cloning: that a neural network, given a well-constructed visual state, can implicitly learn a non-linear control mapping in place of an explicit feedback controller. No classical PID or pure-pursuit comparator was implemented in this study, so this is an architectural rather than a comparative-performance claim. Notably, the learned policy tracked all topologies without the sustained oscillatory steering often anticipated from a feedback law lacking a derivative damping term, suggesting that the CNN learned a temporally coherent steering policy from the expert demonstrations.

Perhaps the most significant and transferable finding of this study is the decisive role of the visual information bottleneck in reducing shortcut learning. The full-AOI allocentric model achieved low validation loss during training yet failed during physical deployment, indicating that the network had learned an unintended decision

rule based on absolute spatial position rather than the geometric features required for robust path following. According to Geirhos et al., shortcut learning occurs when a model exploits features that are sufficient for good performance on training and similarly distributed test data but fail under out-of-distribution conditions. The observed failure of the allocentric representation aligns with this behavior, as the learned strategy did not transfer reliably to real-world operating conditions. By replacing this representation with a semantically abstracted robot-centered crop on a black canvas, the representation removed many of the positional cues that enabled shortcut learning. As a result, the network was encouraged to learn features based on relative path geometry rather than absolute arena position, leading to improved generalization during physical deploymentGeirhos et al. (2020).

This forced the convolutional feature extractor to operate exclusively on relative geometric cues such as the curvature, orientation, and proximity of the approaching path boundaries, which are the only features that generalize across track positions and recovery scenarios. This architectural decision became the definitive factor separating successful real-world deployment from offline training success and represents a replicable design principle for future low-cost end-to-end navigation systems.

More broadly, this study demonstrates that the integration of classical geometric computer vision, specifically projective homography, with modern imitation learning is a viable and computationally efficient strategy for structured indoor robot navigation. By offloading the burden of 3D geometry interpretation from the neural network to a deterministic analytical transformation, the complexity of the learning task was substantially reduced.

A compact DAVE-2 CNN with approximately 2.2 million parameters (2,211,175) was sufficient to learn reliable path-following behaviors that would otherwise require much deeper architectures when operating on raw perspective images. The system's successful operation on a laptop-class GPU, with real-time inference delivered over a local wireless link to a low-cost microcontroller-driven robot, further confirms the feasibility of this approach for educational and resource-constrained industrial applications.

5.3 | Recommendations for Future Work

5.3.1 | Onboard Hardware Acceleration and Latency Reduction

The current architecture offloads all image processing and CNN inference to a remote workstation, with control commands transmitted to the robot over a local Wi-Fi link. This centralized design introduces network-dependent latency that may become problematic at higher robot velocities or in environments with wireless interference.

Future work should explore migrating the inference pipeline directly onto an onboard edge computing platform. Deploying the trained DAVE-2 model on an NVIDIA Jetson Nano or a Google Coral Edge TPU would eliminate the round-trip network delay and increase the effective control loop frequency beyond the measured operating rate of approximately 30 Hz.

Model compression techniques such as post-training quantization (INT8) and structured pruning should also be evaluated, as the simplified BEV input representation suggests that the network may be substantially over-parameterized for the task.

5.3.2 | Dynamic Obstacle Integration

A fundamental limitation of the current Behavioral Cloning approach is its inability to safely handle dynamic obstacles. The training dataset captures human driving in a static environment, and the learned policy has no mechanism for detecting or responding to objects that appear unexpectedly on the track.

Future iterations should investigate integrating the existing vision pipeline with lightweight secondary sensing modalities. Ultrasonic rangefinders or miniature 1D LiDAR sensors mounted on the robot chassis could provide a proximity signal that triggers a rule-based stop or yield behavior operating independently of the CNN controller.

Alternatively, a lightweight object detection module such as MobileNet-SSD running in parallel on the overhead frame could inject obstacle presence as an additional channel in the state representation, allowing the CNN to learn obstacle-aware steering policies through augmented data collection sessions.

5.3.3 | Arena Scaling and Marker-Free Generalization

The current system is constrained to a predefined arena bounded by four fixed ArUco corner markers, limiting its operational area to approximately one square meter in the prototype implementation.

Future work should investigate scaling the homography framework to larger environments through distributed marker networks or the use of higher-resolution cameras with wider fields of view. A more ambitious extension would explore replacing the fiducial-marker homography system with a learned monocular depth estimation module capable of generating approximate BEV representations in environments where physical marker installation is impractical.

5.3.4 | Dataset Augmentation and Robustness to Illumination Variation

The trained model is sensitive to environmental conditions that affect ArUco marker detectability, particularly inconsistent or low-contrast lighting. Because the current training dataset was collected under controlled artificial lighting conditions, the learned policy is not robust to detection failures caused by natural light variation, specular reflections, or partial marker occlusion.

Future data collection protocols should deliberately include sessions under varied lighting conditions, shadow patterns, and partial marker visibility to improve the robustness of the perception pipeline. Additionally, offline data augmentation strategies such as synthetic brightness and contrast jitter applied to the egocentric crops may improve the CNN's tolerance to minor rendering artifacts without requiring additional physical driving sessions.

5.3.5 | Recurrent and Attention-Based Architectures

The current DAVE-2 architecture processes each frame independently as a stateless regression model. This design does not leverage temporal context, such as recent steering history or the observed trajectory of the approaching path over preceding frames.

Integrating a recurrent module such as a Long Short-Term Memory (LSTM) layer appended to the fully connected stack, or a lightweight spatiotemporal attention

mechanism, could enable the network to anticipate upcoming curves based on changes in path curvature rather than reacting solely to the instantaneous visual state.

This is particularly relevant for suppressing the residual oscillatory steering behavior observed at higher throttle speeds, where latency in the single-frame control loop becomes a contributing factor to instability.

5.4 | Final Remarks

This research demonstrates that reliable autonomous navigation can be achieved using a minimal and low-cost vision-based framework when classical geometric transformations are effectively combined with modern imitation learning techniques. The study highlights the importance of carefully designing the robot's visual state representation, particularly in mitigating overfitting and improving sim-to-real transfer performance.

By leveraging homography-based overhead localization and an egocentric semantic perception pipeline, the proposed system achieved stable real-world navigation without relying on expensive sensing hardware or computationally intensive mapping algorithms. The findings of this study contribute toward the development of accessible and scalable autonomous robotic systems suitable for educational, research, and resource-constrained deployment scenarios.

References

Y. I. Abdel-Aziz and H. M. Karara. Direct linear transformation from comparator coordinates into object space coordinates in close-range photogrammetry. *Photogrammetric Engineering & Remote Sensing*, 81(2): 103–107, 2015. doi: 10.14358/PERS.81.2.103.

D. Adapa, V. S. Shekhawat, A. Gautam, and S. Mohan. Autonomous mapping and navigation using fiducial markers and pan-tilt camera for assisting indoor mobility of blind and visually impaired people. In *2019 IEEE 5th International Conference for Convergence in Technology (I2CT)*, pages 1–6, 2019. URL `https://arxiv.org/abs/2310.10290`.

M. S. Alam, A. I. Gullu, and A. Gunes. Fiducial markers and particle filter based localization and navigation framework for an autonomous mobile robot. *Automatika*, 64(3):447–459, 2023. doi: 10.1007/s42979-024-03090-y.

M. Alghamdi, A. Al-Marakeby, and S. Abdel-Mageid. Mobile robot navigation based on artificial markers: A systematic mapping study. *Journal of Robotics and Mechatronics*, 37(3):762–778, 2025. doi: 10.20965/jrm.2025.p0762.

E. Alimovski, G. Erdemir, and A. E. Kuzucuoglu. Vision-based localization in urban areas for mobile robots. *Applied Sciences*, 12(19):9877, 2022. doi: 10.3390/s25041178.

M. Asif, H. W. Lim, Y. Thadimari, N. Imanberdiyev, and E. Camci. Localization of a tethered drone & ground robot team by deep neural networks. In *2020 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, pages 1475–1480, 2020. doi: 10.1109/ITSC58415.2024.10920023.

M. Bansal, A. Krizhevsky, and A. Ogale. Chauffeurnet: Learning to drive by imitating the best and synthesizing the worst. *IEEE Robotics and Automation Letters*, 5(2):374–381, 2020. doi: 10.48550/arXiv.1812.03079.

S. Bhat and A. Kavasseri. Multi-source data integration for navigation in gps-denied autonomous driving environments. *International Journal of Electrical and Electronics Research*, 12(3):863–869, 2024. doi: 10.37391/IJEER.120317.

Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Prasoon Goyal, Lawrence D. Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.

G. Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 25(11):120–125, 2000.

J. Braun, A. O. Júnior, G. Berger, V. H. Pinto, I. N. Soares, A. I. Pereira, J. Lima, and P. Costa. A robot localization proposal for the robotatfactory 4.0: A novel robotics competition within the industry 4.0 concept. *Frontiers in Robotics and AI*, 9:1023590, 2022. doi: 10.3389/frobt.2022.1023590.

D. C. Brown. Decentering distortion of lenses. *Photogrammetric Engineering*, 32(3):444–462, 1966.

C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard. Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. *IEEE Transactions on Robotics*, 32(6):1309–1332, 2016. doi: 10.1109/TRO.2016.2624754.

S. Cebollada, L. Payá, M. Flores, A. Peidró, and O. Reinoso. A state-of-the-art review on mobile robotics tasks using artificial intelligence and visual data. *Expert Systems With Applications*, 167:114195, 2020. doi: 10.1016/j.eswa.2020.114195.

C. P. Chea, Y. Bai, and Z. Zhou. Design and development of robotic collaborative system for automated construction of reciprocal frame structures. *Computer-Aided Civil and Infrastructure Engineering*, 39(10): 1550–1569, 2023. doi: 10.1111/mice.13145.

P. Chen, Z. Luo, X. Jiang, Z. Yin, and J. Li. Maps for autonomous driving: Full-process survey and frontiers. *arXiv*, 2025.

X. Chen, P. Wang, X. Long, J. Gao, J. Lin, and X. Wang. TransTrack: Multiple-object tracking with transformer. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 15039–15049, 2021. doi: 10.48550/arXiv.2012.15460.

K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, 2014. doi: 10.48550/arXiv.1406.1078.

T. Cieslewski, S. Choudhary, and D. Scaramuzza. Data-efficient decentralized visual SLAM. In *2017 IEEE International Conference on Robotics and Automation*, pages 2466–2473, 2017. doi: 10.48550/arXiv.1710.05772.

J. Civera and S. H. Lee. RGB-D odometry and SLAM. In P. Corke and S. Hutchinson, editors, *Robotics and Automation Handbook*, pages 117–144. Springer, 2017. doi: 10.48550/arXiv.2001.06875.

Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). In *4th International Conference on Learning Representations (ICLR)*, 2016.

F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy. Exploring the limitations of behavior cloning for autonomous driving. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 9329–9338, 2019. doi: 10.48550/arXiv.1904.08980.

Y. Dai and K. Lee. Multi-sensor fusion for autonomous mobile robot docking: Integrating LiDAR, YOLO-based AprilTag detection, and depth-aided localization. *Applied Sciences*, 13(6):3467, 2023. doi: 10.3390/electronics14142769.

P. De La Puente, G. Vega-Martínez, P. Javierre, J. Laserna, and E. Martin-Arias. Combining vision and range sensors for AMCL localization in corridor environments with rectangular signs. *Frontiers in Robotics and AI*, 12:1652251, 2025. doi: 10.3389/frobt.2025.1652251.

M. Á. de Miguel, F. García, and J. M. Armingol. Improved LiDAR probabilistic localization for autonomous vehicles using GNSS. *Sensors*, 20(11):3145, 2020. doi: 10.3390/s20113145.

V. De Silva, J. Roche, and A. Kondoz. Robust fusion of LiDAR and wide-angle camera data for autonomous mobile robots. *Sensors*, 18(8):2730, 2018. doi: 10.3390/s18082730.

A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proceedings of the International Conference on Learning Representations*, 2021. doi: 10.48550/arXiv.2010.11929.

M. Dreissig, D. Scheuble, F. Piewak, and J. Boedecker. Survey on LiDAR perception in adverse weather conditions. *arXiv*, 2023. doi: 10.48550/arXiv.2304.06312.

D. Eigen, C. Puhrsch, and R. Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems*, volume 27, pages 2366–2374, 2014. doi: 10.48550/arXiv.1406.2283.

Z. Fang, Y. Chen, M. Zhou, and C. Lu. Marker-based mapping and localization for autonomous valet parking. In *2020 IEEE Intelligent Vehicles Symposium*, pages 1471–1476, 2020. doi: 10.1007/978-981-96-9961-2_35.

Aleksandra Faust, Oscar Ramirez, Marek Fiser, Kenneth Oslund, Anthony G. Francis, James Davidson, and Lydia Tapia. PRM-RL: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning. *CoRR*, abs/1710.03937, 2017.

M. Fiala. ARTag, a fiducial marker system using digital techniques. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, volume 2, pages 590–596, 2005. doi: 10.1109/CVPR.2005.74.

S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez. Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6):2280–2292, 2014. doi: 10.1016/j.patcog.2014.01.005.

S. Gatesichapakorn, J. Takamatsu, and M. Ruchanurucks. ROS based autonomous mobile robot navigation using 2d LiDAR and RGB-D camera. In *2019 First International Symposium on Instrumentation, Control, Artificial Intelligence, and Robotics*, pages 151–154, 2019. doi: 10.1109/ICA-SYMP.2019.8645984.

Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard S. Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, April 2020. doi: 10.1038/s42256-020-00257-z. URL `https://doi.org/10.1038/s42256-020-00257-z`.

I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

R. Han, S. Wang, S. Wang, Z. Zhang, J. Chen, S. Lin, C. Li, C. Xu, Y. C. Eldar, Q. Hao, and J. Pan. NeuPAN: Direct point robot navigation with end-to-end model-based learning. *arXiv*, 2024. doi: 10.48550/arXiv.2403.06828.

R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2003.

S. Hinderer, M. Scheffler, and B. Yang. Investigation of ArUco marker placement for planar indoor localization. *arXiv*, 2025. doi: 10.48550/arXiv.2509.17345.

C. C. Ho and C.-D. Lin. Pose-based visual servoing with lightweight deep-learning binarization for autonomous mobile robot application. In *2020 IEEE International Conference on Systems, Man, and Cybernetics*, pages 3727–3732, 2020. doi: 10.1109/APSIPAASC58517.2023.10317548.

S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

Q. Huang, J. DeGol, V. Fragoso, S. N. Sinha, and J. J. Leonard. Optimizing fiducial marker placement for improved visual localization. *arXiv*, 2022. doi: 10.48550/arXiv.2211.01513.

B. Ichter, J. Harrison, and M. Pavone. Learning sampling distributions for robot motion planning. In *2018 IEEE International Conference on Robotics and Automation*, pages 7087–7094, 2018. doi: 10.48550/arXiv.1709.05448.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

J. Jeon, Y. Hwang, Y. Jeong, S. Park, I. S. Kweon, and S. B. Choi. Lane detection aided online dead reckoning for GNSS denied environments. *Sensors*, 21(20):6805, 2021. doi: 10.3390/s21206805.

A. Jokić, M. Petrović, and Z. Miljković. Neural network-based visual servoing of wheeled mobile robot with fish-eye camera. In *Lecture Notes in Electrical Engineering*, pages 37–49. 2019. doi: 10.1007/978-3-032-02106-9_7.

S. Khan, M. Naseer, M. Hayat, F. S. Khan, and M. Shah. Transformers in vision: A survey. *ACM Computing Surveys*, 54(10):1–41, 2022. doi: 10.1145/3505244.

Y.-H. Kim, J.-I. Jang, and S. Yun. End-to-end deep learning for autonomous navigation of mobile robot. In *2018 IEEE International Conference on Consumer Electronics*, pages 1–6, 2018. doi: 10.1109/ICCE.2018.8326229.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015.

P. Kolar, P. Benavidez, and M. Jamshidi. Survey of data fusion techniques for laser and vision based sensor integration for autonomous navigation. *Sensors*, 20(8):2180, 2020. doi: 10.3390/s20082180.

A. Kostusiak and P. Skrzypczynski. Enhancing visual odometry with estimated scene depth: Leveraging RGB-D data with deep learning. *Sensors*, 23(11):5304, 2023. doi: 10.3390/electronics13142755.

K. Krinkin, K. Chayka, A. Filatov, and A. Filatov. Autonomous wheels and camera calibration in duckietown project. *Procedia Computer Science*, 150:661–667, 2019. doi: 10.1016/j.procs.2021.04.135.

D. Kumar and N. Muhammad. A survey on localization for autonomous vehicles. *IEEE Access*, 11: 115865–115883, 2023. doi: 10.1109/ACCESS.2023.3326069.

Sofiia Kuzmenko. Fiducial markers overview: Types, use cases, & comparison table, May 2025. URL `https://www.it-jim.com/blog/fiducial-markers-types/`. Published May 14, 2025, 14 min read.

Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.

Esko Lehtonen, Otto Lappi, Iivo Koirikivi, and Heikki Summala. Effect of driving experience on anticipatory look-ahead fixations in real curve driving. *Accident Analysis & Prevention*, 2014. doi: 10.1016/j.aap.2014.04.002. URL `http://dx.doi.org/10.1016/j.aap.2014.04.002`.

K. Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2(2):164–168, 1944. doi: 10.1090/qam/10666.

C.-T. Lin. Implementation and evaluation of marker-based visual localization for mobile robots. Master's thesis, Technische Hochschule Ingolstadt, 2023.

Y. Liu, C. Zhao, and Y. Wei. A robust localization system fusion vision-CNN relocalization and progressive scan matching for indoor mobile robots. *Sensors*, 22(5):1845, 2022. doi: 10.3390/app12063007.

Y. Liu, S. Wang, Y. Xie, T. Xiong, and M. Wu. A review of sensing technologies for indoor autonomous mobile robots. *Sensors*, 24(4):1222, 2024. doi: 10.3390/s24041222.

Y. Lu, H. Ma, E. Smart, and H. Yu. Real-time performance-focused localization techniques for autonomous vehicle: A review. *IEEE Transactions on Intelligent Transportation Systems*, 23(7):6082–6100, 2021. doi: 10.1109/TITS.2021.3077800.

Z. Machkour, D. Ortiz-Arroyo, and P. Durdevic. Monocular based navigation system for autonomous ground robots using multiple deep learning models. *Journal of Intelligent & Robotic Systems*, 106(2):38, 2022. doi: 10.1007/s44196-023-00250-5.

B. R. K. Mantha and B. G. De Soto. Investigating the fiducial marker network characteristics for autonomous mobile indoor robot navigation using ROS and gazebo. *Journal of Construction Engineering and Management*, 148(10), 2022. doi: 10.1061/(ASCE)CO.1943-7862.0002378.

F. Massa, L. Bonamini, A. Settimi, L. Pallottino, and D. Caporale. LiDAR-based GNSS denied localization for autonomous racing cars. *Sensors*, 20(14):3992, 2020. doi: 10.3390/s20143992.

V. Mohanty, S. Agrawal, S. Datta, A. Ghosh, V. D. Sharma, and D. Chakravarty. DeepVO: A deep learning approach for monocular visual odometry. *arXiv*, 2016. doi: 10.48550/arXiv.1611.06069.

S. Montecchio. Study and development of a reliable fiducials-based localization system for multicopter UAVs flying indoor. Master's thesis, University of Padova, 2022.

R. Muñoz-Salinas, M. J. Marín-Jimenez, and R. Medina-Carnicer. SPM-SLAM: Simultaneous localization and mapping with squared planar markers. *Pattern Recognition*, 86:156–171, 2018. doi: 10.1016/j.patcog.2017.08.010.

M. Nahangi, A. Heins, B. McCabe, and A. Schoellig. Automated localization of UAVs in GPS-denied indoor construction environments using fiducial markers. In *Proceedings of the 35th International Symposium on Automation and Robotics in Construction*, pages 18–25, 2018. doi: 10.22260/ISARC2018/0012.

J. S. Oh and M. Y. Kim. Regression-based docking system for autonomous mobile robots using a monocular camera and ArUco markers. *Sensors*, 25(12):3742, 2025. doi: 10.3390/s25123742.

E. Olson. AprilTag: A robust and flexible visual fiducial system. In *2011 IEEE International Conference on Robotics and Automation*, pages 3400–3407, 2011. doi: 10.1109/ICRA.2011.5979561.

T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, and J. Peters. An algorithmic perspective on imitation learning. *Foundations and Trends in Robotics*, 7(1–2):1–179, 2018. doi: 10.1561/2300000053.

Christos Papachristos, Frank Mascarich, and Kostas Alexis. Thermal-inertial localization for autonomous navigation of aerial robots through obscurants. *International Conference on Unmanned Aircraft Systems*, pages 394–399, June 2018. doi: 10.1109/icuas.2018.8453447.

I. Park and S. Cho. Fusion localization for indoor airplane inspection using visual inertial odometry and ultrasonic RTLS. *Scientific Reports*, 13(1):18117, 2023. doi: 10.1038/s41598-023-43425-y.

C. Patruno, V. Renò, M. Nitti, N. Mosca, M. di Summa, and E. Stella. Vision-based omnidirectional indoor robots for autonomous navigation and localization in manufacturing industry. *Applied Soft Computing*, 87:105986, 2020. doi: 10.1016/j.heliyon.2024.e26042.

B. Pfrommer and K. Daniilidis. TagSLAM: Robust SLAM with fiducial markers. *arXiv*, 2019. doi: 10.48550/arXiv.1910.00679.

J. M. Pierre. End-to-end deep learning for robotic following. In *Proceedings of the 2018 2nd International Conference on Mechatronics Systems and Control Engineering*, pages 77–85. Association for Computing Machinery, 2018.

K. Radha and S. Karthikeyan. A hybrid recurrent neural network and optimization framework for intelligent mobile robot navigation in smart manufacturing. *Scientific Reports*, 15:28710, 2025.

J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016. doi: 10.48550/arXiv.1506.02640.

E. A. Rodríguez-Martínez, W. Flores-Fuentes, F. Achakir, O. Sergiyenko, and F. N. Murrieta-Rico. Vision-based navigation and perception for autonomous robots: Sensors, SLAM, control strategies, and cross-domain applications—a review. *Eng*, 6(7):153, 2025. doi: 10.3390/eng6070153.

F. J. Romero-Ramírez, R. Muñoz-Salinas, and R. Medina-Carnicer. Speeded up detection of squared fiducial markers. *Image and Vision Computing*, 76:38–47, 2018. doi: 10.1016/j.imavis.2018.05.004.

O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pages 234–241. Springer, 2015. doi: 10.48550/arXiv.1505.04597.

A. Rosebrock. Detecting ArUco markers with OpenCV and Python. PyImageSearch, 2024. URL `https://pyimagesearch.com`.

J. Rosner, T. Krzeszowski, A. Świtoński, H. Josiński, W. Lindenheim-Locher, M. Zieliński, G. Paleta, M. Paszkuta, and K. Wojciechowski. Multimodal dataset for indoor 3D drone tracking. *Scientific Data*, 12(1):257, 2025. doi: 10.1038/s41597-025-04521-y.

F. Salem and O. S. Gedik. Thermal SLAM: Harnessing temperature variations to enhance object detection and tracking performance. *Journal of Smart Systems Research*, 8(2):558–568, 2025. doi: 10.34248/bsengineering.1578563.

A. Sevgi and H. E. Koçer. Multicamera-based indoor localization and path optimization for mobile robots using ArUco markers. *El-Cezeri Fen Ve Mühendislik Dergisi*, 12(3):339–355, 2025. doi: 10.21203/rs.3.rs-5720392/v1.

W. K. Sleaman, A. A. Hameed, and A. Jamil. Monocular vision with deep neural networks for autonomous mobile robots navigation. *Journal of Intelligent Systems*, 31(1):975–990, 2022. doi: 10.1016/j.ijleo.2022.170162.

R. Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2010.

S. Tenzin, A. Rassau, and D. Chai. Application of event cameras and neuromorphic computing to VSLAM: A survey. *IEEE Sensors Journal*, 24(18):27913–27929, 2024. doi: 10.3390/biomimetics9070444.

T. Thepsit, P. Konghuayrob, A. Saenthon, and S. Yanyong. Localization for outdoor mobile robot using LiDAR and RTK-GNSS/INS. In *2023 20th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology*, pages 1–5, 2023.

S. Tolenov and B. Omarov. Real-time self-localization and mapping for autonomous navigation of mobile robots in unknown environments. *International Journal of Advanced Computer Science and Applications*, 15 (1), 2024. doi: 10.14569/IJACSA.2024.0151090.

A. Tourani, H. Bavle, J. L. Sanchez-Lopez, R. M. Salinas, and H. Voos. Marker-based visual SLAM leveraging hierarchical representations. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 3461–3467, 2023. doi: 10.48550/arXiv.2303.01155.

I. Ullah, X. Su, J. Zhu, X. Zhang, D. Choi, and Z. Hou. Evaluation of localization by extended kalman filter, unscented kalman filter, and particle filter-based techniques. *Wireless Communications and Mobile Computing*, 2020:8898672, 2020. doi: 10.1155/2020/8898672.

A. Vina. Understanding the integration of computer vision in robotics. *International Journal of Robotics Research*, 42(5):234–256, 2023.

Zhoubo Wang, Zhenhai Zhang, Wei Zhu, Xuehai Hu, Hongbin Deng, Guang He, and Xiao Kang. A robust planar marker-based visual SLAM. *Sensors*, 23(2):917, 2023. doi: 10.3390/s23020917.

A. Waqas and Y.-J. Cha. Fiducial marker-based localization of autonomous UAV for structural health monitoring. *Structural Health Monitoring*, 21(6):2728–2743, 2022. doi: 10.1007/978-3-031-34938-6_18.

L. Wei, H. Sun, G. Qiao, X. Wu, and H. Xu. A review of SLAM research on mobile robot vision. *Frontiers in Science and Engineering*, 4(11):17–23, 2024. doi: 10.54691/gcwwng31.

X. Weng, K. Kitani, and J. Gottschlich. Baseline and benchmark for motion forecasting using recurrent architectures. *IEEE Robotics and Automation Letters*, 4(3):695–702, 2019.

J. Whitaker, R. Christensen, and G. Droge. Global localization of ground vehicles using self-describing fiducials coupled with IMU data. In *2020 IEEE/ION Position, Location and Navigation Symposium*, pages 186–196, 2020. doi: 10.1109/PLANS46316.2020.9109829.

X. Xiao, B. Liu, G. Warnell, and P. Stone. Motion planning and control for mobile robot navigation using machine learning: A survey. *Autonomous Robots*, 46(5):569–597, 2022. doi: 10.1007/s10514-022-10039-8.

W. Xu, D. Yang, J. Liu, Y. Li, and M. Zhou. A visual navigation algorithm for UAV based on visual-geography optimization. *Drones*, 7(2):127, 2023. doi: 10.3390/drones8070313.

K. Yokoyama and K. Morioka. Autonomous mobile robot with simple navigation system based on deep reinforcement learning and a monocular camera. *Advanced Robotics*, 34(17):1113–1125, 2020. doi: 10.1109/SII46433.2020.9025987.

T. Yu, Q. Zhang, and T. Liu. Reinforcement learning approaches in the motion systems of autonomous underwater vehicles. *Applied Ocean Research*, 161:104682, 2025. doi: 10.1016/j.apor.2025.104682.

K. E. Yulianto, R. E. Saputro, and F. S. Utomo. Lightweight visual detection system for object identification with ArUco markers in resource-constrained environments. *Electronic Journal of Education Social Economics and Technology*, 6(2):757–768, 2025.

Y. Zhang, A. Carballo, H. Yang, and K. Takeda. Perception and sensing for autonomous vehicles under adverse weather conditions: A survey. *ISPRS Journal of Photogrammetry and Remote Sensing*, 196:146–177, 2023. doi: 10.1016/j.isprsjprs.2022.12.021.

Z. Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000. doi: 10.1109/34.888718.

Y. Zhou and W. Zhang. End-to-end robot intelligent obstacle avoidance method based on deep reinforcement learning with spatiotemporal transformer architecture. *Frontiers in Neurorobotics*, 19: 1646336, 2025. doi: 10.3389/fnbot.2025.1646336.

Y. Zhou, Z. Yu, and Z. Ma. UAV based indoor localization and object detection. *Frontiers in Neurorobotics*, 16:914353, 2022. doi: 10.3389/fnbot.2022.914353.

J. Zhu, H. Li, and T. Zhang. Camera, LiDAR, and IMU based multi-sensor fusion SLAM: A survey. *Tsinghua Science & Technology*, 29(2):415–429, 2023. doi: 10.26599/TST.2023.9010010.

Calendar of Activities

Table 1: Project Implementation Schedule and Timeline

Phase	Activity	Timeline
I. Hardware Setup		
	Hardware & system configuration	Jan Week 1–2
	Chassis assembly & wiring	Jan Week 2–3
	Camera mounting & calibration	Jan Week 2–3
II. Software Development		
	OS & environment setup	Jan Week 3–4
	Dev: perception (homography)	Jan Week 4–Feb Week 6
	Dev: data logger module	Feb Week 6–7
III. Data Pipeline		
	Dataset collection (driving)	Feb Week 7–Mar Week 9
	Data cleaning & augmentation	Mar Week 10–11
IV. Model Development		
	CNN architecture design & dev.	Mar Week 12–13
	Model training (GPU)	Apr Week 13–14
	Hyperparameter tuning	Apr Week 14–15
V. Integration & Testing		
	Deployment to robot	May Week 14–15
	Scenario testing (A, B, C)	May Week 15–17
	Analysis & documentation	May Week 17

Personal Vitae

John Christian Niño T. Abuel

- Title: 4th Year Bachelor of Science in Computer Science
- Affiliation: Mindanao State University – Iligan Institute of Technology
- Address: Block 2 Lot 27 Doña Maria Subdivision Phase 5, Tipanoy, Iligan City
- Contact Number: 0945 702 5852
- Email Address: johnchristiannino.abuel@g.msuiit.edu.ph

Kervin Lemuel Paalisbo

- Title: 4th Year Bachelor of Science in Computer Science
- Affiliation: Mindanao State University – Iligan Institute of Technology
- Address: Poblacion Linamon Purok 5-A, Lanao Del Norte
- Contact Number: 09924779137
- Email Address: kervinlemuel26@gmail.com

Rey Iann Tigley

- Title: 4th Year Bachelor of Science in Computer Science

- Affiliation: Mindanao State University – Iligan Institute of Technology
- Address: Prk Sudlonon, San Miguel, Iligan City, 9200 Lanao del Norte
- Contact Number: 09605052173
- Email Address: reyiann.tigley@g.msuiit.edu.ph